

Zuul Rapport de Projet : Pokémon Delta Emerald

Site Web : <https://perso.esiee.fr/~jouanarb/>

Groupe : A3P 2025/2026 G5

I.A) Auteur : Barnabe Jouanard

I.B) Thème

Un jeu d'aventure se déroulant dans la région de Hoenn (Pokémon), axé sur l'exploration et la résolution d'énigmes environnementales pour mettre fin à un conflit légendaire entre la terre et la mer.

I.C) Résumé du Scénario

Le joueur incarne un dresseur à Littleroot Town. Le monde subit un climat anormal. L'objectif principal est de traverser la partie ouest de Hoenn pour trouver l'Orbe Émeraude (cachée à Petalburg City) et l'apporter au Pilier Céleste pour réveiller Rayquaza.

I.D) Plan

La carte se compose de 9 lieux principaux :

Your House: Starting location.

Littleroot Town: Connecting hub.

Route 101: Path to the north.

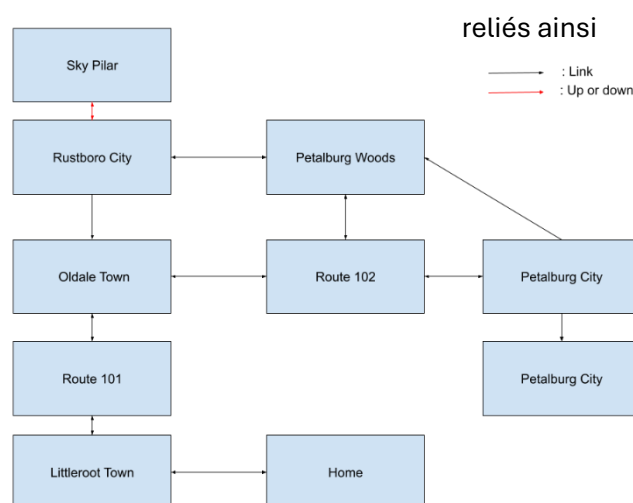
Oldale Town: Junction to the west.

Route 102: Combat/Trainer route.

Petalburg City: City containing a locked Gym.

Sky Pillar: The final destination.

Cascade: Transporter Room



I.E) Scenario Detaille

Le joueur commence dans sa Maison. Il doit se rendre à Little Root Town puis sur la Route 101. Il se dirige vers Oldale Town Petalburg City, ou il devra trouver l'orbe Delta. Le joueur devra trouver un chemin à travers les Bois Petalburg pour atteindre le Pilier Céleste et combattre Rayquaza.

I.F) Détail des items et des personnages.

Dresseurs : Javier, Zucko, Ichigo et Rayquaza

Objets : - Chaussures : Pour plus de flow.

- Map : Pour voir la carte.
- Delta Orb : pour invoquer Rayquaza.
- Grant : Une bourse contenant de l'argent
- Wallet : Un portefeuille contenant de l'argent

I.G) Conditions de Victoire/Défaite

Victoire : Atteindre le sommet du Pilier Céleste en possédant l'Orbe Émeraude pour combattre Rayquaza.

Défaite : Un compteur de tours simulant l'aggravation de la météo. Si le compteur atteint zéro, la région est détruite.

I.I) Commentaires

La version Final a toutes les fonctionnalités auquel j'ai pensé au départ.

II. Réponses aux Exercices

Exercice 7.5

J'ai implémenté la méthode privée printLocationInfo() dans Game pour centraliser l'affichage de la description de la pièce et des sorties. Cela évite la duplication de code entre printWelcome() et goRoom().

```
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print("Exits : ");
    if (this.aCurrentRoom.aNorthExit != null){
        System.out.print("North ");
    }
    if (this.aCurrentRoom.aSouthExit != null){
        System.out.print("South ");
    }
    if (this.aCurrentRoom.aEastExit != null){
        System.out.print("East ");
    }
    if (this.aCurrentRoom.aWestExit != null){
        System.out.print("West ");
    }
}
```

Exercice 7.6 :

J'ai passé les attributs de la classe Room en private et créé l'accesseur getExit pour masquer les détails internes des sorties. Cette modification réduit le couplage en permettant à la classe Game de demander une direction sans accéder directement aux variables de Room.

```
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

```
public Room getExit(final String pDirection){  
    if (pDirection.equals("North"))  
    {  
        return aNorthExit;  
    }  
    if (pDirection.equals("South"))  
    {  
        return aSouthExit;  
    }  
    if (pDirection.equals("East"))  
    {  
        return aEastExit;  
    }  
    if (pDirection.equals("West"))  
    {  
        return aWestExit;  
    }  
    return null;  
}
```

Exercice 7.7 :

J'ai créé la méthode getExitString dans Room pour déléguer la préparation de la liste des sorties à la pièce elle-même. Cela permet à Game d'afficher les informations sans avoir à connaître la structure interne des sorties de Room.

```
public String getExitString()  
{  
    String vReturnString = "Exits:";  
  
    if (this.getExit("North") != null) {  
        vReturnString += " North";  
    }  
  
    if (this.getExit("East") != null) {  
        vReturnString += " East";  
    }  
  
    if (this.getExit("South") != null) {  
        vReturnString += " South";  
    }  
  
    if (this.getExit("West") != null) {  
        vReturnString += " West";  
    }  
  
    return vReturnString;  
} // getExitString()
```

```
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print(this.aCurrentRoom.getExitString());
    System.out.println();
}
```

Exercice 7.8 :

J'ai remplacé les attributs de direction par une `HashMap<String, Room>` pour centraliser la gestion des sorties. J'ai adapté `getExitString`, `printLocationInfo`, `getExit` et le constructeur pour `HashMap` et aussi réécrit dans la classe `game` les sorties pour adapter ces changements.

```
private HashMap<String, Room> aExits;
```

```
public Room (final String pDescription)
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
} // Room()
```

```
public Room getExit(final String pDirection)
{
    return this.aExits.get(pDirection);
} // getExit()
```

```
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.aExits.get("North") != null) {
        vReturnString += " North";
    }

    if (this.aExits.get("East") != null) {
        vReturnString += " East";
    }

    if (this.aExits.get("South") != null) {
        vReturnString += " South";
    }

    if (this.aExits.get("West") != null) {
        vReturnString += " West";
    }
}
```

```
public void setExit (final String pDirection,final Room pExit)
{
    this.aExits.put(pDirection, pExit);
} // setExits()
```

```
vHouse.setExit("West", vLittleroot);

vLittleroot.setExit("North", vRoute101);
vLittleroot.setExit("East", vHouse);

vRoute101.setExit("North", vOldale);
vRoute101.setExit("South", vLittleroot);

vOldale.setExit("East", vRoute102);
vOldale.setExit("South", vRoute101);

vRoute102.setExit("North", vPetalburgWoods);
vRoute102.setExit("East", vPetalburg);
vRoute102.setExit("West", vOldale);

vPetalburg.setExit("South", vRoute102);
vPetalburg.setExit("West", vRustboro);

vPetalburgWoods.setExit("South", vRoute102);
vPetalburgWoods.setExit("West", vRustboro);
```

Exercice 7.8.1 :

J'ai étendu la gestion des déplacements en ajoutant les directions verticales "up" et "down", permettant ainsi d'accéder au Sky Pillar. Pour cela, j'ai simplement utilisé la nouvelle méthode setExit pour lier les pièces et mis à jour getExitString pour afficher ces sorties supplémentaires ainsi que goRoom pour pouvoir se déplacer.

```
public String getExitString()
{
    String vReturnString = "Exits: ";
    if (this.aExits.get("North") != null) {
        vReturnString += " North";
    }
    if (this.aExits.get("East") != null) {
        vReturnString += " East";
    }
    if (this.aExits.get("South") != null) {
        vReturnString += " South";
    }
    if (this.aExits.get("West") != null) {
        vReturnString += " West";
    }
    if (this.aExits.get("Down") != null) {
        vReturnString += " Down";
    }
    if (this.aExits.get("Up") != null) {
        vReturnString += " Up";
    }
    return vReturnString;
} // getExitString()

else if (pCommand.getSecondWord().equals("Up"))
{
    if (this.aCurrentRoom.getExit("Up") != null)
    {
        this.aCurrentRoom = this.aCurrentRoom.getExit("Up");
        this.printLocationInfo();
    } else
    {
        System.out.println("There is no door !");
    }
}
else if (pCommand.getSecondWord().equals("Down"))
{
    if (this.aCurrentRoom.getExit("Down") != null)
    {
        this.aCurrentRoom = this.aCurrentRoom.getExit("Down");
        this.printLocationInfo();
    } else
    {
        System.out.println("There is no door !");
    }
}

vRustboro.setExit("Up", vSkyPillar);
vRustboro.setExit("East", vPetalburgWoods);
vSkyPillar.setExit("Down", vRustboro);
```

Exercice 7.9 :

J'ai remplacé la vérification manuelle des directions dans getExitString par une boucle for-each parcourant l'ensemble des clés (keySet) de la HashMap. Cette modification permet de construire dynamiquement la liste des sorties disponibles, rendant la classe Room capable d'afficher n'importe quelle nouvelle direction (comme "up" ou "down") sans aucune modification de code supplémentaire.

```

public String getExitString()
{
    String vReturnString = "Exits: ";
    Set<String> vKeys = this.aExits.keySet(); // Récupère toutes les directions présentes
    for (String vExit : vKeys) {
        vReturnString += " " + vExit;
    } // for
    return vReturnString;
} // getExitString()

```

Exercice 7.11 :

J'ai implémenté la méthode `getLongDescription()` dans la classe `Room` afin qu'elle assemble elle-même la description du lieu et la liste des sorties. La méthode `printLocationInfo()` de la classe `Game` a été simplifiée pour n'appeler que cette fonction, masquant ainsi totalement les détails d'implémentation interne de la pièce au reste de l'application.

```

public String getLongDescription()
{
    return "You are " + this.getDescription() + '\n' + this.getExitString();
}

private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getLongDescription());
} // printLocationInfo()

```

Exercice 7.14 :

J'ai créé la commande `look` qui permet au joueur d'afficher la description complète du lieu actuel et de ses sorties sans effectuer de déplacement. Pour améliorer l'expérience utilisateur, j'ai également mis à jour la liste des mots de commande dans `CommandWords` afin que `look` apparaisse lors de l'appel à la commande `help`.

```

private final String[] aValidCommands = { "go", "help", "quit", "look" };

```

```

else if (pCommand.getCommandWord().equals("look")){
    look();
    return false;
}

```

```

private void look()
{
    System.out.println(this.aCurrentRoom.getLongDescription());
} //look()

```

```

private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println(" go quit help look");
} // printHelp()

```

Exercice 7.15 :

J'ai ajouté la commande `eat` qui, une fois activée, affiche un message indiquant que le joueur a mangé et n'a plus faim. Techniquement, cette commande a été enregistrée dans la classe `CommandWords` et intégrée à la structure de contrôle de la méthode `processCommand` dans la classe `Game`.

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat" };

    else if (pCommand.getCommandWord().equals("eat")){
        eat();
        return false;
    }

private void eat()
{
    System.out.println("You have eaten now and you are not hungry any more.");
} //eat()

private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println(" go quit help look eat");
} // printHelp()
```

Exercice 7.16 :

J'ai délégué la responsabilité de l'affichage des commandes à la classe CommandWords via une méthode showAll(), appelée par le Parser. Ce changement permet à la commande help de mettre à jour automatiquement sa liste de mots (incluant look et eat) sans qu'aucune modification ne soit nécessaire dans la classe Game.

```
public void showAll(){
    for(String vCommand : aValidCommands){
        System.out.print(vCommand + ' ');
    }
    System.out.println();
} // showAll()

public void showCommands(){
    this.aValidCW.showAll();
} // showCommands()

private void printHelp()
{
    System.out.println("You are in the wonderful word of Pokemon.");
    System.out.println("You are trying to stop Rayquaza from destroying Hoenn.");
    System.out.println("Your command words are:");
    aParser.showCommands();
} // printHelp()
```

Exercice 7.18:

La méthode showAll a été modifiée en getCommandList pour renvoyer une chaîne de caractères contenant les commandes, et les méthodes showCommand (Parser) et printHelp (Game) ont été adaptées en conséquence. Suite à une comparaison avec "Zuul better", la classe StringBuilder a été intégrée aux méthodes getExitString et getCommandList pour optimiser la construction des chaînes de caractères, et des problèmes de formatage ont été corrigés.

```
public String getCommandList(){
    String vChaineDesCommandes = "";
    for(String vCommand : aValidCommands){
        vChaineDesCommandes = vChaineDesCommandes + vCommand + " ";
    }
    return vChaineDesCommandes;
} // showAll()

public String getCommandsList()
{
    return this.aValidCW.getCommandList();
} // showCommands()

private void printHelp()
{
    System.out.println("You are in the wonderful word of Pokemon.");
    System.out.println("You are trying to stop Rayquaza from destroying Hoenn.");
    System.out.println("Your command words are:");
    System.out.println(aParser.getCommandsList());
} // printHelp()
```

Exercice 7.18.1 a 7.18.4:

Le titre du jeu reste "Pokémon Delta Emerald". Des images ont été choisies pour les différentes pièces du jeu. La classe Room a été mise à jour avec un attribut `imageName` et un accesseur `getImageName()` pour gérer ces illustrations.

```
public String getExitString()
{
    StringBuilder vReturnString = new StringBuilder("Exits:");
    for (String vExit : this.aExits.keySet()) {
        vReturnString.append(" ").append(vExit);
    }
    return vReturnString.toString();
} // getExitString()

public String getCommandList()
{
    StringBuilder vChaineDesCommandes = new StringBuilder();
    for (String vCommand : this.aValidCommands) {
        vChaineDesCommandes.append(vCommand).append(" ");
    }
    return vChaineDesCommandes.toString();
} // getCommandList()
```

Exercice 7.18.6:

Le projet est passé sur la base "zuul with image", ce qui a impliqué plusieurs changements et adaptations dans le code. Par exemple, la classe Room a été adaptée avec l'ajout de l'attribut `imageName`, de l'accesseur `getImageName()`, et la modification du constructeur pour prendre en charge et afficher ces illustrations.

```
private String aImageName;

public String getImageName()
{
    return this.aImageName;
} // getImageName()

public Room (final String pDescription, final String pImage)
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
} // Room()

public String getImageName()
{
    return this.aImageName;
} // getImageName()
```



```

private CommandWords aValidCW; // (voir la classe CommandWords)

/**
 * Constructeur par défaut qui cree les 2 objets prevus pour les attributs
 */
public Parser()
{
    this.aValidCW = new CommandWords();
    // System.in designe le clavier, comme System.out designe l'ecran
} // Parser()

/**
 * Get a new command from the user. The command is read by
 * parsing the 'inputLine'.
 */
public Command getCommand( final String pInputLine )
{
    String vWord1;
    String vWord2;

    StringTokenizer tokenizer = new StringTokenizer( pInputLine );

    if ( tokenizer.hasMoreTokens() )
        vWord1 = tokenizer.nextToken(); // get first word
    else
        vWord1 = null;

    if ( tokenizer.hasMoreTokens() )
        vWord2 = tokenizer.nextToken(); // get second word
    else
        vWord2 = null;

    // note: we just ignore the rest of the input line.

    // Now check whether this word is known. If so, create a command
    // with it. If not, create a "null" command (for unknown command).

    if ( this.aValidCW.isCommand( vWord1 ) )
        return new Command( vWord1, vWord2 );
    else
        return new Command( null, vWord2 );
} // getCommand()

/**
 * Print out a list of valid command words.
 */
public String getCommandString()
{
    return this.aValidCW.getCommandList();
} // getCommandString()

private UserInterface aGui;
private GameEngine aEngine;

/**
 * Default constructor: initializes the engine and the gui.
 */
public Game()
{
    this.aEngine = new GameEngine();
    this.aGui = new UserInterface( this.aEngine );
    this.aEngine.setGUI( this.aGui );
} // Game()

public void setGUI( final UserInterface pUserInterface )
{
    this.aGui = pUserInterface;
    this.printWelcome();
}

private void printWelcome()
{
    this.aGui.print("\n");
    this.aGui.println("Welcome to the World of Pokemon!");
    this.aGui.println("A Wonderful World Where You Can Live An Adventure!");
    this.aGui.println("Type 'help' if you need help.");
    this.aGui.print("\n");
    this.printLocationInfo();
    if ( this.aCurrentRoom.getImageName() != null )
        this.aGui.showImage( this.aCurrentRoom.getImageName() );
} // printWelcome()

```

```

private void createRooms()
{
    Room house, littleroot, route101, oldale, route102, petalburg, petalburgWoods, rustboro, skyPillar;

    house = new Room("in your house in Littleroot Town","Littleroot_Town.jpg");
    littleroot = new Room("in your Littleroot, your Home Town","Littleroot_Town.jpg");
    route101 = new Room("on Route 101, wild Pokemon might appear in the tall grass","Littleroot_Town.jpg");
    oldale = new Room("in Oldale Town, a small junction with a Pokemon Center","Littleroot_Town.jpg");
    route102 = new Room("on Route 102, wild Pokemon might appear in the tall grass","Littleroot_Town.jpg");
    petalburg = new Room("in Petalburg City, where your father is the Gym Leader","Littleroot_Town.jpg");
    petalburgWoods = new Room("in Petalburg Woods","Littleroot_Town.jpg");
    rustboro = new Room("in Rustboro City, where the Devon Corporation is located","Littleroot_Town.jpg");
    skyPillar = new Room("on the Sky Pillar, Rayquaza's home..","Sky Pillar.jpg");

    house.setExit("west", littleroot);

    littleroot.setExit("north", route101);
    littleroot.setExit("east", house);

    route101.setExit("north", oldale);
    route101.setExit("south", littleroot);

    oldale.setExit("east", route102);
    oldale.setExit("south", route101);

    route102.setExit("north", petalburgWoods);
    route102.setExit("east", petalburg);
    route102.setExit("west", oldale);

    petalburg.setExit("north", petalburgWoods);
    petalburg.setExit("west", route102);

    petalburgWoods.setExit("south", route102);
    petalburgWoods.setExit("west", rustboro);

    rustboro.setExit("up", skyPillar);
    rustboro.setExit("east", petalburgWoods);
    rustboro.setExit("south", oldale);

    skyPillar.setExit("down", rustboro);

    this.aCurrentRoom = house;
} // createRooms()

```

```

public void processCommand (final String pCommand)
{
    this.aGui.println( "> " + pCommand );
    Command vCommand = this.aParser.getCommand( pCommand );

    if ( vCommand.isUnknown() ) {
        this.aGui.println( "I don't know what you mean..." );
        return;
    }

    String vCommandWord = vCommand.getCommandWord();
    if ( vCommandWord.equals( "help" ) )
        this.printHelp();
    else if ( vCommandWord.equals( "go" ) )
        this.goRoom( vCommand );
    else if ( vCommandWord.equals( "eat" ) )
        this.eat();
    else if ( vCommandWord.equals( "look" ) )
        this.look();
    else if ( vCommandWord.equals( "quit" ) ) {
        if ( vCommand.hasSecondWord() )
            this.aGui.println( "Quit what?" );
        else
            this.endGame();
    }
} // processCommand()

```

```

private void endGame()
{
    this.aGui.println( "Thank you for playing. Good bye." );
    this.aGui.enable( false );
}

```

```

private void look()
{
    this.aGui.println(this.aCurrentRoom.getLongDescription());
} //look()

```

```

/**
 * Prints a message indicating the player has eaten.
 */
private void eat()
{
    this.aGui.println("You have eaten now and you are not hungry any more.");
} //eat()

```

```

private void printLocationInfo()
{
    this.aGui.print(this.aCurrentRoom.getLongDescription());
} // printLocationInfo()

```

```

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.net.URL;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

```

Exercice 7.18.8:

Huit boutons cliquables ont été ajoutés à l'interface graphique (six pour les directions, un pour help, et un pour quit). Les boutons ont été instanciés, ajoutés à un JPanel avec un GridLayout, et reliés à un ActionListener (via la méthode actionPerformed) pour exécuter les commandes dans le moteur du jeu.

```

private JButton aButtonGoNorth;
private JButton aButtonGoSouth;
private JButton aButtonGoEast;
private JButton aButtonGoWest;
private JButton aButtonGoUp;
private JButton aButtonGoDown;
private JButton aButtonQuit;
private JButton aButtonHelp;

```

```

this.aButtonGoNorth = new JButton( "go north" );
this.aButtonGoSouth = new JButton( "go south" );
this.aButtonGoEast = new JButton( "go east" );
this.aButtonGoWest = new JButton( "go west" );
this.aButtonGoUp = new JButton( "go up" );
this.aButtonGoDown = new JButton( "go down" );
this.aButtonQuit = new JButton( "quit" );
this.aButtonHelp = new JButton( "help" );

```

```

vPanel.add( vButtonPanel, BorderLayout.WEST );

```

```

JPanel vButtonPanel = new JPanel();
vButtonPanel.setLayout( new GridLayout( 8, 1 ) );
vButtonPanel.add( this.aButtonGoNorth );
vButtonPanel.add( this.aButtonGoSouth );
vButtonPanel.add( this.aButtonGoEast );
vButtonPanel.add( this.aButtonGoWest );
vButtonPanel.add( this.aButtonGoUp );
vButtonPanel.add( this.aButtonGoDown );
vButtonPanel.add( this.aButtonHelp );
vButtonPanel.add( this.aButtonQuit );

```

```

this.aEntryField.addActionListener( this );
this.aButtonGoNorth.addActionListener( this );
this.aButtonGoSouth.addActionListener( this );
this.aButtonGoEast.addActionListener( this );
this.aButtonGoWest.addActionListener( this );
this.aButtonGoDown.addActionListener( this );
this.aButtonGoUp.addActionListener( this );
this.aButtonHelp.addActionListener( this );
this.aButtonQuit.addActionListener( this );

```

```

@Override public void actionPerformed( final ActionEvent pE )
{
    if( this.aButtonGoNorth.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go north");
    }
    else if( this.aButtonGoSouth.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go south");
    }
    else if( this.aButtonGoEast.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go east");
    }
    else if( this.aButtonGoWest.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go west");
    }
    else if( this.aButtonGoUp.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go up");
    }
    else if( this.aButtonGoDown.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go down");
    }
    else if( this.aButtonQuit.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("quit");
    }
    else if( this.aButtonHelp.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("help");
    }
    else {
        this.processCommand();
    }
}

```

Exercice 7.19.2:

Déplacement de toutes les images dans le sous dossier /images.

```

Room house, littleroot, route101, oldale, route102, petalburg, petalburgWoods, rustboro, skyPillar;

house      = new Room("in your house in Littleroot Town","images/home.jpg");
littleroot = new Room("in your Littleroot, your Home Town","images/littleroot town.jpg");
route101   = new Room("on Route 101, wild Pokemon might appear in the tall grass","images/route 101.jpg");
oldale     = new Room("in Oldale Town, a small junction with a Pokemon Center","images/oldaletown.jpg");
route102   = new Room("on Route 102, wild Pokemon might appear in the tall grass","images/route 102.jpg");
petalburg  = new Room("in Petalburg City, where your father is the Gym Leader","images/petalburg city.jpg");
petalburgWoods = new Room("in Petalburg Woods","images/petalburg woods.jpg");
rustboro   = new Room("in Rustboro City, where the Devon Corporation is located","images/rustboro city.jpg");
skyPillar  = new Room("on the Sky Pilar, Rayquaza's home..","images/sky pillar.jpg");

```

Exercice 7.20:

La classe Item a été créée avec les attributs aDescription et aPrice. L'implémentation initiale dans la classe Room permettait de stocker un seul objet et la méthode getLongDescription a été retravaillée pour afficher cet objet s'il est présent.

```

private String aDescription;
private int aPrice;

public Item (final String pDescription,final int pPrice)
{
    aDescription = pDescription;
    aPrice = pPrice;
}

public String getItemString()
{
    return this.aDescription;
} // getItemString()

public int getItemPrice()
{
    return this.aPrice;
} // getItemPrice()

```

```
private Item aItem;
```

```

public void setItem(final Item pItem)
{
    this.aItem = pItem;
}

public Item getItem(){
    return this.aItem;
}

```

```

house.setItem(new Item("map",0));
skyPillar.setItem(new Item("Delta Orb",100));

```

```

public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(this.aItem != null){
        vDescription = vDescription + '\n' + "There is " +
            this.aItem.getItemString() + '\n' + "Item price : " + this.aItem.getItemPrice();
    }
    else {
        vDescription = vDescription + '\n' + "there is no Items here";
    }
    return vDescription;
} // getLongDescription()

```

Exercice 7.21:

La méthode getItemString a été remplacée par getItemDescription pour renvoyer une description plus complète incluant le poids (ou prix) de l'objet, ce qui a été intégré dans getLongDescription.

```

public String getItemDescription()
{
    return "there is " + this.aDescription + "Item weight : " + this.aPrice;
}

```

```

public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(this.aItem != null){
        vDescription = vDescription + '\n' + this.aItem.getItemDescription();
    }
    else {
        vDescription = vDescription + '\n' + "there is no Items here";
    }
    return vDescription;
} // getLongDescription()

```

Exercice 7.22:

La classe Room a été modifiée pour utiliser une HashMap<String, Item> nommée aItems, permettant à une pièce de contenir plusieurs objets. La méthode getLongDescription itère désormais sur cette collection pour afficher tous les objets présents.

```

private HashMap<String, Item> aItems;

```

```

public Item getItem(final String pName){
    return this.aItems.get(pName);
}

```

```

public void addItem(final String pName, final Item pItem) {
    this.aItems.put(pName, pItem);
}

```

```

public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(!this.aItems.isEmpty()){
        for (String vName : this.aItems.keySet()) {
            vDescription += "\n" + this.aItems.get(vName).getItemDescription();
        }
    }
    else {
        vDescription += '\n' + "There are no Items here!";
    }
    return vDescription;
} // getLongDescription()

```

Exercice 7.22.2:

Plusieurs objets ont été instanciés au démarrage du jeu : une "Map" et des "Shoes" dans la maison (House), ainsi que la "Delta Orb" dans la pièce du Sky Pillar.

```

house.addItem("Map",new Item("The Map",0));
house.addItem("Shoes",new Item("Your Shoes",0));
skyPillar.addItem("Delta Orb",new Item("The Delta Orb, the orb that will allow you to summon Rayquazza..",100));

```

Exercice 7.23:

Ajout de la commande back permettant au joueur de revenir sur ses pas. Un historique des déplacements a d'abord été implémenté avec une `LinkedList<Room>` (`aPreviousRoom`). Des conditions ont été ajoutées pour empêcher le joueur de faire "back" s'il vient de commencer.

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back" };
```

```
private LinkedList<Room> aPreviousRoom = new LinkedList<Room>();
```

```
this.aPreviousRoom.add(null);
```

```
private void goRoom (final Command pCommand)
{
    if ( ! pCommand.hasSecondWord() ) {
        // if there is no second word, we don't know where to go...
        this.aGui.println( "Go where?" );
        return;
    }

    String vDirection = pCommand.getSecondWord();

    // Try to leave current room.
    Room vNextRoom = this.aCurrentRoom.getExit( vDirection );

    if ( vNextRoom == null )
        this.aGui.println( "There is no door!" );
    else {
        this.aPreviousRoom.addLast(this.aCurrentRoom);
        this.aCurrentRoom = vNextRoom;
        this.aGui.println( this.aCurrentRoom.getLongDescription() );
        if ( this.aCurrentRoom.getImageName() != null )
            this.aGui.showImage( this.aCurrentRoom.getImageName() );
    }
} // goRoom()
```

```
else if ( vCommandWord.equals( "back" ) )
    if ( vCommand.hasSecondWord() )
        this.aGui.println( "Back what?" );
    else
        if ( this.aPreviousRoom.getLast() != null )
            this.goBack();
        else
            this.aGui.println( "Back where? You just Started!" );
```

```
private void goBack()
{
    this.aCurrentRoom = this.aPreviousRoom.getLast();
    this.aPreviousRoom.removeLast();
    this.printLocationInfo();
    this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

Exercice 7.26:

La `LinkedList` a été remplacée de manière plus judicieuse par une pile `Stack<Room>` (`aPreviousRooms`), et les méthodes `goRoom` et `goBack` ont été mises à jour pour utiliser `push()` et `pop()`.

```
private Stack<Room> aPreviousRooms = new Stack<Room>();
```

```
this.aPreviousRooms.push(this.aCurrentRoom);
```

```
if (!this.aPreviousRooms.isEmpty())
```

```
private void goBack()
{
    this.aCurrentRoom = this.aPreviousRooms.pop(); // retrieves AND removes the top
    this.printLocationInfo();
    this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

Exercice 7.28.1:

Une commande test a été ajoutée pour lire et exécuter automatiquement une suite de commandes depuis un fichier texte (avec l'extension .txt). Cette fonctionnalité s'appuie sur `InputStream` et `Scanner` pour lire les lignes et les envoyer à `interpretCommand`.

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test" };
```

```
import java.io.InputStream;
import java.util.Scanner;
```

```
else if (vCommandWord.equals("test"))
{
    if(vCommand.hasSecondWord() == false)
        this.aGui.println("Need a file name!");
    else
        test(vCommand);
}
```

```
private void test(final Command pCommand)
{
    String vFileName = pCommand.getSecondWord();
    vFileName = vFileName + ".txt";

    InputStream vIS = this.getClass().getClassLoader().getResourceAsStream(vFileName);

    if (vIS == null)
    {
        System.out.println("File not found: " + vFileName);
        return;
    }

    Scanner vSC = new Scanner(vIS);

    while (vSC.hasNextLine())
    {
        String vLigne = vSC.nextLine();
        this.interpretCommand(vLigne);
    }

    vSC.close();
}
```

Exercice 7.29:

Création de la classe `Player` pour encapsuler les données du joueur, notamment son nom (`aName`), sa position actuelle (`aCurrentRoom`), et son historique de pièces (`aPreviousRooms`). Le `GameEngine` a été adapté en conséquence et une commande `name` a été introduite pour permettre au joueur de se renommer.

```
private String aName;
private Room aCurrentRoom;
private Stack<Room> aPreviousRooms;
```

```
/**
 * Create a player with a name and a starting room.
 * @param pName the player name (can be empty or null)
 * @param pStartRoom the initial room
 */
```

```
public Player(final String pName, final Room pStartRoom)
```

```
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
}
```

```
/**
 * Return the player's current room.
 */
```

```
public Room getCurrentRoom()
{
    return this.aCurrentRoom;
}
```

```

/**
 * Change the current room, saving the previous one.
 * This is called by the GameEngine after it validates the direction.
 * @param pNextRoom the room to move to
 */

```

```

public void moveTo(final Room pNextRoom)

```

```

{
    if (this.aCurrentRoom != null) {
        this.aPreviousRooms.push(this.aCurrentRoom);
    }
    this.aCurrentRoom = pNextRoom;
}

```

```

/**
 * Return true if the player can go back to a previous room.
 */

```

```

public boolean canGoBack()
{
    return !this.aPreviousRooms.isEmpty();
}

```

```

/**
 * Move back to the previous room if possible.
 * @return the new current room (or current room if no previous room)
 */

```

```

public void goBack()
{
    this.aCurrentRoom = this.aPreviousRooms.pop();
}

```

```

/**
 * Return the player name.
 */

```

```

public String getName()
{
    return this.aName;
}

```

```

/**
 * Set the player name.
 */

```

```

public void setName(final String pName)
{
    this.aName = pName;
}

```

```

private void goRoom( final Command pCommand )

```

```

{
    if ( ! pCommand.hasSecondWord() ) {
        // If there is no second word, we don't know where to go.
        this.aGui.println( "Go where?" );
        return;
    }

```

```

    String vDirection = pCommand.getSecondWord();

```

```

    Room vNextRoom = this.aPlayer.getCurrentRoom().getExit( vDirection );

```

```

    if ( vNextRoom == null )
        this.aGui.println( "There is no door!" );
    else {

```

```

        this.aPlayer.moveTo(vNextRoom);
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
        if ( this.aPlayer.getCurrentRoom().getImageName() != null )
            this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    }
} // goRoom()

```

```

private void goBack( final Command pCommand )

```

```

{
    if ( pCommand.hasSecondWord() ) {
        // If there is a second word, we don't know where to go.
        this.aGui.println( "Back where?" );
        return;
    }

```

```

    if (!this.aPlayer.canGoBack())
        this.aGui.println( "Can't go back, you just started!" );
    else {
        this.aPlayer.goBack();
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
        if ( this.aPlayer.getCurrentRoom().getImageName() != null )
            this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    }
}

```

```

} // goRoom()

```

```

this.aPlayer = new Player("Luke", house);

```

```

else if (vCommandWord.equals("name"))

```

```

{
    if(vCommand.hasSecondWord() == false)
        this.aGui.println("You need a name!");
    else
        name(vCommand);
}

```

```

private void name(final Command pCommand)

```

```

{
    String vName = pCommand.getSecondWord();
    this.aPlayer.setName(vName);
    this.aGui.println( "Your new name is " + this.aPlayer.getName() + "!" );
    this.printWelcome();
} // name()

```


Exercice 7.30:

Ajout des commandes take et drop. Le joueur pouvait initialement ramasser et stocker un seul objet via des méthodes setItem et removeItem.

```
private String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test", "name", "take", "drop"};
```

```
private Item aItem;
```

```
public void setItem(final Item pItem)
{
    this.aItem = pItem;
}
```

```
public void removeItem(final String pName){
    this.aItems.remove(pName);
}
```

```
public Item getItem()
{
    return this.aItem;
}
```

```
public void removeItem(){
    this.aItem = null;
}
```

```
private void take(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false)
    {
        this.aGui.println("Take what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);

    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.setItem(vItem);
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
    this.aGui.println("Took " + vItemName + "!");
}
```

```
private void drop (final Command pCommand){
    if(pCommand.hasSecondWord() == false){
        this.aGui.println("Drop what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getItem();
    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem();
    this.aGui.println("Dropped " + vItemName + "!");
}
```

```
else if ( vCommandWord.equals( "take" ) )
    this.take(vCommand);
else if ( vCommandWord.equals( "drop" ) )
    this.drop(vCommand);
```

Exercice 7.31:

Le joueur est désormais capable de transporter plusieurs objets, son inventaire étant géré par une HashMap<String, Item>. Les méthodes take et drop ont été mises à jour pour utiliser addItem et removeItem à la fois pour le joueur et la pièce.

```
private HashMap<String, Item> aItems;
```

```
public Player(final String pName, final Room pStartRoom)
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new HashMap<String, Item>();
}
```

```

public Item getItem(final String pNom)
{
    return this.aItems.get(pNom);
}

public void addItem(final String pNom, final Item pItem)
{
    this.aItems.put(pNom, pItem);
}

public void removeItem(final String pNom)
{
    this.aItems.remove(pNom);
}

private void take(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false)
    {
        this.aGui.println("Take what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);

    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
    this.aPlayer.addItem(vItemName, vItem);
    this.aGui.println("Took " + vItemName + "!");
}

private void drop(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false){
        this.aGui.println("Drop what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getItem(vItemName);
    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem(vItemName);
    this.aGui.println("Dropped " + vItemName + "!");
}

```

Exercice 7.31.1:

Pour éviter la duplication de code entre Player et Room, la classe ItemList a été créée. Elle encapsule la logique de gestion de la HashMap des objets (addItem, getItem, removeItem, getItemsDescription). Les classes Room et Player l'utilisent désormais comme attribut (aItems).

```
private HashMap<String, Item> aItems;

/**
 * Creates an empty item list.
 */
public ItemList()
{
    this.aItems = new HashMap<String, Item>();
}

/**
 * Returns the item associated with the given name, or {@code null}.
 *
 * @param pName the lookup key
 * @return the matching {@link Item}, or {@code null}
 */
public Item getItem( final String pName )
{
    return this.aItems.get( pName );
}

/**
 * Adds or replaces an item under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pItem the {@link Item} to store
 */
public void addItem( final String pName, final Item pItem )
{
    this.aItems.put( pName, pItem );
}

/**
 * Removes the item associated with the given key.
 *
 * @param pName the lookup key
 */
public void removeItem( final String pName )
{
    this.aItems.remove( pName );
}

/**
 * Returns {@code true} if the list contains no items.
 */
public boolean isEmpty()
{
    return this.aItems.isEmpty();
}

/**
 * Builds a multi-line string with each item's description,
 * or a default notice when the list is empty.
 *
 * @return formatted item descriptions
 */
public String getItemsDescription()
{
    if ( this.aItems.isEmpty() ) {
        return "There are no items here!";
    }
    String vResult = "";
    for ( String vName : this.aItems.keySet() ) {
        vResult += this.aItems.get( vName ).getItemDescription() + "\n" ;
    }
    return vResult;
}

private ItemList aItems;

/**
 * Creates a room with a description, image path, and empty exit/item maps.
 *
 * @param pDescription human-readable location phrase shown to the player
 * @param pImage resource path for the picture, or {@code null} if none
 */
public Room( final String pDescription, final String pImage )
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
    this.aItems = new ItemList();
} // Room()

public Item getItem( final String pName )
{
    return this.aItems.getItem( pName );
}

/**
 * Places (or replaces) an item in this room under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pItem the {@link Item} instance to store
 */
public void addItem( final String pName, final Item pItem )
{
    this.aItems.addItem( pName, pItem );
}

public void removeItem(final String pName){
    this.aItems.removeItem(pName);
}
```

```
private ItemList aItems;

/**
 * Create a player with a name and a starting room.
 * @param pName the player name (can be empty or null)
 * @param pStartRoom the initial room
 */
public Player(final String pName, final Room pStartRoom)
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new ItemList();
}
```

```
public Item getItem(final String pNom)
{
    return this.aItems.getItem(pNom);
}
```

```
public void addItem(final String pNom, final Item pItem)
{
    this.aItems.addItem(pNom, pItem);
}
```

```
public void removeItem(final String pNom)
{
    this.aItems.removeItem(pNom);
}
```

Bonus :

Pour mettre en place la fin du jeu, un attribut booléen `isWinningRoom` a été ajouté pour déterminer si une pièce spécifique est la salle déclenchant la victoire. Des méthodes d'accès ont été intégrées pour manipuler et vérifier cet état.

Modification de la classe `Room` : Ajout d'un setter pour définir la pièce comme gagnante et d'un booléen public (getter) pour vérifier son état.

Définition de l'objectif : Lors de la création des pièces, le `Sky Pillar` est défini comme la salle de victoire.

Vérification dans le `GameEngine` : Une condition a été ajoutée pour vérifier si le joueur se trouve dans la winning room tout en possédant l'objet requis ("Delta Orb"). Si ces conditions sont remplies, le joueur gagne la partie.

```
public void setAsWinningRoom()
{
    this.isWinningRoom = true;
}
```

```
public boolean isWinningRoom()
{
    return this.isWinningRoom;
}
```

```
skyPillar.setAsWinningRoom();
```

```
if ( this.aPlayer.getItem( "delta-orb" ) != null ) {
    if ( this.aPlayer.getCurrentRoom().isWinningRoom() ) {
        this.aGui.println( "\n" + "Congratulations! You just won! Thank you for playing the whole game!" );
    }
}
```

Un système d'affichage de carte évolutif a été ajouté, permettant au joueur de voir sa position à condition d'avoir préalablement ramassé l'objet "carte". Adaptation de la classe `Room` : Le constructeur de la pièce accepte désormais un paramètre supplémentaire pour lier une image de carte spécifique (`aMapImageName`) au lieu. Chaque pièce est désormais instanciée avec son image principale et l'image de sa carte (ex: `images/home_map.jpeg`). Remaniement de l'interface graphique (`UserInterface`) : Mise en page de l'image : Création d'un nouveau panel exclusif à l'image principale pour la maintenir centrée en utilisant `setHorizontalAlignment` et `setVerticalAlignment`. Zone de navigation : Les boutons de déplacement et d'action (incluant le nouveau bouton `back`) ont été organisés sous forme de grille 3x3 (`GridLayout(3, 3)`). Zone de la carte : L'affichage de la carte a été assigné à la position est (`BorderLayout.EAST`) du panneau

inférieur (SouthPanel). Logique interactive dans le GameEngine : Le système vérifie si le joueur possède la carte lors des déplacements (goRoom, goBack) et lorsqu'il ramasse des objets (take) afin d'actualiser l'affichage instantanément. Si le joueur a la carte, la méthode affiche l'image de la carte associée à la pièce actuelle. Si le joueur ne possède pas ou lâche la carte (drop), une image par défaut (ex: images/no_map.jpeg) est affichée à la place.

```
private String aMapImageName;

public Room( final String pDescription, final String pImage, final String pMapImage )
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
    this.aMapImageName = pMapImage;
    this.aItems = new ItemList();
} // Room()

house = new Room( "in your house in Littleroot Town", "images/home.gif", "images/home_map.jpeg" );
littleroot = new Room( "in Littleroot Town, your home town", "images/littleroot town.gif", "images/littleroot_map.jpeg" );
route101 = new Room( "on Route 101; wild Pokémon might appear in the tall grass", "images/route 101.gif", "images/route101_map.jpeg" );
oldale = new Room( "in Oldale Town, a small junction with a Pokémon Center", "images/oldale town.gif", "images/oldale town_map.jpeg" );
route102 = new Room( "on Route 102; wild Pokémon might appear in the tall grass", "images/route 102.gif", "images/route102_map.jpeg" );
petalburg = new Room( "in Petalburg City, where your father is the Gym Leader", "images/petalburg city.gif", "images/petalburg city_map.jpeg" );
petalburgWoods = new Room( "in Petalburg Woods", "images/petalburg woods.gif", "images/petalburg woods_map.jpeg" );
rustboro = new Room( "in Rustboro City, where the Devon Corporation is located", "images/rustboro city.gif", "images/rustboro city_map.jpeg" );
skyPillar = new Room( "on the Sky Pillar, Rayquaza's home", "images/sky pillar.gif", "images/sky pilar_map.jpeg" );

this.aImage = new JLabel();
this.aImage.setHorizontalAlignment( JLabel.CENTER );
this.aImage.setVerticalAlignment( JLabel.CENTER );

this.aMap = new JLabel();

this.aButtonGoUp = new JButton( "go up" );
this.aButtonGoNorth = new JButton( "go north" );
this.aButtonGoDown = new JButton( "go down" );
this.aButtonGoWest = new JButton( "go west" );
this.aButtonGoEast = new JButton( "go east" );
this.aButtonBack = new JButton( "go back" );
this.aButtonHelp = new JButton( "help" );
this.aButtonGoSouth = new JButton( "go south" );
this.aButtonQuit = new JButton( "quit" );

JPanel vButtonPanel = new JPanel();
vButtonPanel.setLayout( new GridLayout( 3, 3 ) );
vButtonPanel.add( this.aButtonGoUp );
vButtonPanel.add( this.aButtonGoNorth );
vButtonPanel.add( this.aButtonGoDown );
vButtonPanel.add( this.aButtonGoWest );
vButtonPanel.add( this.aButtonGoEast );
vButtonPanel.add( this.aButtonBack );
vButtonPanel.add( this.aButtonHelp );
vButtonPanel.add( this.aButtonGoSouth );
vButtonPanel.add( this.aButtonQuit );

JPanel vPanel = new JPanel();
vPanel.setLayout( new BorderLayout() );
vPanel.add( this.aImage, BorderLayout.CENTER );

JPanel vSouthPanel = new JPanel();
vSouthPanel.setLayout( new BorderLayout() );
vSouthPanel.setPreferredSize( new Dimension( 0, 400 ) );

vSouthPanel.add( vListScroller, BorderLayout.CENTER );
vSouthPanel.add( this.aEntryField, BorderLayout.SOUTH );
vSouthPanel.add( vButtonPanel, BorderLayout.WEST );
vSouthPanel.add( this.aMap, BorderLayout.EAST );

vPanel.add(vSouthPanel, BorderLayout.SOUTH);

this.aMyFrame.getContentPane().add( vPanel, BorderLayout.CENTER );

this.aEntryField.addActionListener( this );
this.aButtonGoNorth.addActionListener( this );
this.aButtonGoSouth.addActionListener( this );
this.aButtonGoEast.addActionListener( this );
this.aButtonGoWest.addActionListener( this );
this.aButtonGoUp.addActionListener( this );
this.aButtonGoDown.addActionListener( this );
this.aButtonBack.addActionListener( this );
this.aButtonQuit.addActionListener( this );
this.aButtonHelp.addActionListener( this );
```

```
if ( this.aPlayer.getCurrentRoom().getImageName() != null ) {
    this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    if ( this.aPlayer.getItem( "map" ) != null ) {
        this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
    }
}
```

```
if ( this.aPlayer.getItem( "map" ) != null ) {
    this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
}
```

```
if ( this.aPlayer.getItem( "map" ) == null ) {
    this.aGui.showMap( "images/no_map.jpeg" );
}
```

Exercice 7.32:

Un système de monnaie a été mis en place, rendant l'argent nécessaire pour l'obtention des objets. Classe Player : Un nouvel attribut entier aMoney a été créé pour représenter le solde du joueur. Cet attribut s'accompagne de ses méthodes d'accès, à savoir un accesseur getMoney() et un mutateur setMoney(). Par défaut, le joueur commence avec 150 unités de monnaie. Classe GameEngine : Ce système a été implémenté au sein des méthodes take et drop. Lors de la saisie d'un objet via take, le moteur vérifie si le joueur possède un montant supérieur au prix de l'objet ; si c'est le cas, l'objet est ajouté à l'inventaire et le montant est déduit du solde du joueur. À l'inverse, lorsqu'un objet est relâché, le prix de ce dernier est restitué au joueur. S'il n'a pas les fonds nécessaires, un message indique que l'objet est trop cher.

```
private int aMoney;

/**
 * Creates a player with a name and a starting room.
 *
 * @param pName      the player name (can be empty or {@code null})
 * @param pStartRoom the initial room
 */
public Player( final String pName, final Room pStartRoom )
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new ItemList();
    this.aMoney = 150;
}

public int getMoney()
{
    return this.aMoney;
}

public void setMoney( final int pNewMoney )
{
    this.aMoney = pNewMoney;
}

if ( this.aPlayer.getMoney() >= vItem.getItemPrice() )
{
    this.aPlayer.setMoney( this.aPlayer.getMoney() - vItem.getItemPrice() );
    this.aPlayer.getCurrentRoom().removeItem( vItemName );
    this.aPlayer.addItem( vItemName, vItem );
    this.aGui.println( "\nTook " + vItemName + "!" );
    if ( this.aPlayer.getItem( "map" ) != null ) {
        this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
    }
} else
{
    this.aGui.println( vItemName + " is too expensive for you right now !" );
}

this.aPlayer.setMoney( this.aPlayer.getMoney() + vItem.getItemPrice() );
```

Exercice 7.33:

La possibilité pour le joueur de consulter les objets qu'il transporte a été ajoutée via la nouvelle commande inventory. Déclaration : Le mot-clé "inventory" a été ajouté au tableau aValidCommands dans la classe CommandWords. Classe ItemList : La méthode getItemList() a été créée pour générer une chaîne de caractères listant les objets du joueur, en précisant qu'il n'a rien si sa liste est vide. Cette méthode calcule également la valeur totale (vInventoryPrice) des objets possédés. Liaison et Affichage : Une méthode getItemInventory() a été mise en place pour relayer cette information vers la classe Player. Enfin, la méthode inventory() dans le GameEngine se charge d'afficher ce message à l'écran.

```
private String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test", "name", "take", "drop", "inventory" };
```

```

public String getItemList()
{
    if ( this.aItems.isEmpty() ) {
        return "You don't have any items!";
    }
    String vResult = "Your items: ";
    int vInventoryPrice = 0;
    for ( String vItem : this.aItems.keySet() ) {
        vResult += " " + vItem;
        vInventoryPrice += this.aItems.get( vItem ).getItemPrice();
    }
    vResult += "\n" + "\n" + "Total value: " + vInventoryPrice;
    return vResult;
}

public String getItemInventory()
{
    return this.aItems.getItemList() ;
}

private void inventory()
{
    this.aGui.println( this.aPlayer.getItemInventory() );
}

```

Exercice 7.34:

Le jeu intègre désormais un moyen de gagner de l'argent en convertissant certains objets. Refonte de la commande : L'ancienne commande eat a été transformée en la commande cashin dans les CommandWords. Méthode cashin dans GameEngine : La méthode d'origine a été remplacée en profondeur par cashin(final Command pCommand). Objets convertibles : Une HashMap<String, Integer> nommée vCashableItem a été mise en place pour stocker plusieurs objets utilisables pour obtenir de l'argent et leurs valeurs de récompense associées. Par exemple, l'objet "grant" rapporte 75 unités de monnaie, et le "wallet" en rapporte 25. Si le joueur utilise la commande avec un objet valide et qu'il le possède dans son inventaire, l'objet est supprimé et l'argent est crédité sur son solde (setMoney(this.aPlayer.getMoney() + vReward)). Un message confirme ensuite la transaction et affiche le nouveau solde.

```

private void cashin( final Command pCommand )
{
    String vItemName = pCommand.getSecondWord();
    HashMap<String, Integer> vCashableItem = new HashMap<String, Integer>();
    vCashableItem.put( "grant", 75 );
    vCashableItem.put( "wallet", 25 );

    if ( vCashableItem.containsKey( vItemName ) ) {
        Item vItem = this.aPlayer.getItem( vItemName );
        if (vItem != null) {
            int vReward = vCashableItem.get( vItemName );
            this.aPlayer.setMoney(this.aPlayer.getMoney() + vReward);
            this.aPlayer.removeItem( vItemName );
            this.aGui.println("You have cashed in your " + vItemName + " and are now richer!");
            this.aGui.println("Your current money is: " + this.aPlayer.getMoney());
        } else {
            this.aGui.println("You don't have a " + vItemName + " to cash in.");
        }
    } else {
        this.aGui.println("You can't cash that in.");
    }
} // cashin()

```

Exercice 7.34:

Le jeu intègre désormais un moyen de gagner de l'argent en convertissant certains objets. L'ancienne commande eat a été remplacée par la commande cashin. Dans la classe GameEngine, la méthode d'origine a été réécrite pour utiliser une `HashMap<String, Integer>` nommée `vCashableItem`. Cette structure stocke les objets convertibles et leur valeur respective (par exemple, "grant" rapporte 75 unités, "wallet" en rapporte 25). Si le joueur utilise la commande avec un objet valide présent dans son inventaire, l'objet est supprimé et l'argent est crédité sur son solde via `setMoney(this.aPlayer.getMoney() + vReward)`. Un message de confirmation affiche ensuite le nouveau solde.

```
public void interpretCommand( final String pCommand )
{
    this.aGui.println( "\n\n> " + pCommand + "\n" );
    Command vCommand = this.aParser.getCommand( pCommand );
    if ( vCommand.isUnknown() ) {
        this.aGui.println( "I don't know what you mean..." );
        return;
    }
    String vCommandWord = vCommand.getCommandWord();
    switch ( vCommandWord ) {
        case "help":
            this.printHelp();
            break;
        case "go":
            this.goRoom( vCommand );
            break;
        case "cashin":
            this.cashin( vCommand );
            break;
        case "look":
            this.look();
            break;
        case "take":
            this.take( vCommand );
            break;
        case "drop":
            this.drop( vCommand );
            break;
        case "back":
            this.goBack( vCommand );
            break;
        case "inventory":
            this.inventory( vCommand );
            break;
        case "test":
            this.test( vCommand );
            break;
        case "name":
            this.name( vCommand );
            break;
        case "quit":
            this.endGame( vCommand );
            break;
    }
}
// interpretCommand()
```

```
if ( ! pCommand.hasSecondWord() ) {
    this.aGui.println( "You need something to cash in!" );
    return;
}
```

Exercice 7.42:

Afin d'ajouter un enjeu au jeu (Game Over), j'ai implémenté une limite de temps représentée par une `JProgressBar` intégrée à la `UserInterface`. Dans le `GameEngine`, la méthode `losingTimer()` gère ce décompte via une boucle `for` qui retire un pourcentage à la barre toutes les secondes (en utilisant `Thread.sleep(1000)`). Si la barre atteint 0, le jeu affiche une image spécifique ("lose_screen.png") et se ferme automatiquement via la nouvelle méthode `endGame()`.

```
this.aProgressBar = new JProgressBar( 0, 100 );
this.aProgressBar.setStringPainted( true );
this.aProgressBar.setPreferredSize( new Dimension( 200, 50 ) );
this.aProgressBar.setForeground( new Color( 76, 175, 80 ) );
this.aProgressBar.setBackground( new Color( 50, 50, 60 ) );
this.aProgressBar.setValue( 100 );
```



```
vPanel.add( aProgressBar, BorderLayout.NORTH );
```

```
public void setProgress( final int pPercent )
{
    this.aProgressBar.setValue( pPercent );
} // setProgress()
```

```
public int getProgressBar( )
{
    return this.aProgressBar.getValue();
} // getProgressBar()
```

```
private void losingTimer()
{
    for ( int vI = 100; vI >= 0; vI -= 1 ) {
        try { Thread.sleep( 1000 ); } catch ( InterruptedException ignore ) {}
        this.aGui.setProgress( this.aGui.getProgressBar() - 1 );
    }
    this.aGui.println( "Time's up! You lose!" );
    this.aGui.showImage( "assets/lose_screen.png" );
    this.endGame();
} // losingTimer()
```

```
private void endGame()
{
    try { Thread.sleep( 1000 ); } catch ( InterruptedException ignore ) {}
    this.aGui.enable( false );
    try { Thread.sleep( 2000 ); } catch ( InterruptedException ignore ) {}
    System.exit( 0 );
} // quit()
```

Exercice 7.42.2:

Pour améliorer l'esthétique globale de l'IHM, j'ai appliqué une couleur de fond personnalisée (new Color(30, 30, 40)) aux panneaux de l'interface.

```
import java.awt.Color;
```

```
vPanel.setBackground( new Color( 30, 30, 40 ) );
```

Exercice 7.43:

J'ai mis en place un système de portes à sens unique (trapdoor). Pour empêcher le joueur d'utiliser abusivement la commande back dans ce cas précis, j'ai ajouté la méthode isExit(Room) dans la classe Room. Cette méthode confirme si la pièce précédente fait bien partie des sorties valides de la pièce actuelle. Dans le GameEngine, la gestion du retour arrière vérifie désormais cette condition avec getPreviousRoom() ; si la porte d'origine a disparu, le déplacement est annulé.

```
public boolean isExit(final Room pRoom) {
    return this.aExits.containsValue(pRoom);
}
```

```
public Room getPreviousRoom() {
    return this.aPreviousRooms.peek();
}
```

```
} else {
    this.aPlayer.goBack();
    this.aGui.println(this.aPlayer.getCurrentRoom().getLongDescription());
    if (this.aPlayer.getCurrentRoom().getImageName() != null) {
        this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());
        if (this.aPlayer.getItem("map") != null) {
            this.aGui.showMap(this.aPlayer.getCurrentRoom().getMapImageName());
        }
    }

    if (this.aPlayer.getItem("delta-orb") != null && this.aPlayer.getCurrentRoom().isWinningRoom()) {
        this.aGui.println("\n" + "You want to keep winning, don't you?");
    }
}
```

Exercice 7.46:

Pour ajouter un aspect imprévisible, j'ai créé la classe `TransporterRoom`, qui est une extension (héritage) de la classe `Room`. Elle dispose d'un tableau des lieux de destination possibles (`aTargetRooms`). La méthode `getRandomRoom()` utilise la classe `Random` pour renvoyer aléatoirement l'une de ces pièces. Le `GameEngine` a été adapté : la méthode `goRoom` vérifie si la pièce actuelle possède le flag `isTeleporterRoom()` ; si c'est le cas, elle ignore la direction demandée et téléporte le joueur.

```
import java.util.Random;
import java.util.ArrayList;
```

```
/**
 * A special type of {@link Room} that teleports the player to a random location
 *
 * @name Barnabe Jouanard
 * @version 2026.05.01
 */
public class TransporterRoom extends Room
{
    private Room[] aTargetRooms;

    public TransporterRoom(final String pDescription, final String pImage , final String pMapImage, final ArrayList<Room> pTargetRooms)
    {
        super( pDescription, pImage , pMapImage );
        this.aTargetRooms = pTargetRooms.toArray(new Room[0]);
    }

    public Room getRandomRoom()
    {
        Random vRandom = new Random();
        return aTargetRooms[vRandom.nextInt(this.aTargetRooms.length)];
    }
}

/**
 * Marks this room as a teleporter room, which teleports the player to a random location when entered.
 */
public void setAsTeleporterRoom() {
    this.isTeleporterRoom = true;
}

/**
 * Indicates whether this room is flagged as a teleporter room.
 *
 * @return {@code true} if the room is configured as a teleporter room
 */
public boolean isTeleporterRoom() {
    return this.isTeleporterRoom;
}

this.aTeleportableRooms = new ArrayList<Room>();
Room house, littleroot, route101, oldale, route102, petalburg, petalburgWoods, rustboro, skyPillar;
TransporterRoom cascade;

house = new Room("in your house in Littleroot Town", "home.gif", "home_map.jpeg");
this.aTeleportableRooms.add(house);
littleroot = new Room("in Littleroot Town, your home town", "littleroot town.gif", "littleroot_map.jpeg");
this.aTeleportableRooms.add(littleroot);
route101 = new Room("on Route 101; wild Pokémon might appear in the tall grass", "route 101.gif", "route 101_map.jpeg");
this.aTeleportableRooms.add(route101);
oldale = new Room("in Oldale Town, a small junction with a Pokémon Center", "oldale town.gif", "oldale town_map.jpeg");
this.aTeleportableRooms.add(oldale);
route102 = new Room("on Route 102; wild Pokémon might appear in the tall grass", "route 102.gif", "route 102_map.jpeg");
this.aTeleportableRooms.add(route102);
petalburg = new Room("in Petalburg City, where your father is the Gym Leader", "petalburg city.gif", "petalburg city_map.jpeg");
this.aTeleportableRooms.add(petalburg);
petalburgWoods = new Room("in Petalburg Woods", "petalburg woods.gif", "petalburg woods_map.jpeg");
this.aTeleportableRooms.add(petalburgWoods);
rustboro = new Room("in Rustboro City, where the Devon Corporation is located", "rustboro city.gif", "rustboro city_map.jpeg");
this.aTeleportableRooms.add(rustboro);
skyPillar = new Room("on the Sky Pillar, Rayquaza's home", "sky pillar.gif", "sky pillar_map.jpeg");
```

```

cascade = new TransporterRoom("in the cascade's vortex! Who knows where you'll end up?", "cascade.gif");

cascade.setAsTeleporterRoom();

```

```

if (vNextRoom == null && !this.aPlayer.getCurrentRoom().isTeleporterRoom()) {
    this.aGui.println("There is no door!");
} else {
    if (this.aPlayer.getCurrentRoom().isTeleporterRoom()) {
        TransporterRoom vTransporterRoom = (TransporterRoom)this.aPlayer.getCurrentRoom();
        this.aPlayer.moveTo(vTransporterRoom.getRandomRoom());
    } else {
        this.aPlayer.moveTo(vNextRoom);
    }
}

```

Exercice 7.46.1:

La TransporterRoom posant problème pour automatiser les tests, j'ai ajouté la commande alea. Associée à l'attribut aForcedRoom et à la méthode setForcedRoom() dans la TransporterRoom, elle permet de contourner le comportement aléatoire et de fixer la prochaine destination. Pour éviter que le joueur ne l'utilise pour tricher en jeu, j'ai lié cette fonctionnalité au booléen alstest : la commande alea n'est interprétée que si une séquence de test automatisée (via la commande test) est en cours d'exécution.

```

private String[] aValidCommands = {
    "go",
    "help",
    "quit",
    "look",
    "cashin",
    "back",
    "test",
    "alea",
    "name",
    "take",
    "drop",
    "inventory"
};

```

```

case "alea":
    this.alea(vCommand);
    break;

private Room aForcedRoom;

public TransporterRoom(final String pDescription, final String pImage, final String pMapImage, final ArrayList<Room>
{
    super(pDescription, pImage, pMapImage);
    this.aTargetRooms = pTargetRooms.toArray(new Room[0]);
    this.aForcedRoom = null;
}

public Room getRandomRoom()
{
    if (this.aForcedRoom != null) {
        return this.aForcedRoom;
    }
    Random vRandom = new Random();
    return aTargetRooms[vRandom.nextInt(this.aTargetRooms.length)];
}

public void setForcedRoom(final String pRoom){
    switch (pRoom){
        case "house":
            this.aForcedRoom = this.aTargetRooms[0];
            break;
        case "littleroot":
            this.aForcedRoom = this.aTargetRooms[1];
            break;
        case "route101":
            this.aForcedRoom = this.aTargetRooms[2];
            break;
        case "oldale":
            this.aForcedRoom = this.aTargetRooms[3];
            break;
        case "route102":
            this.aForcedRoom = this.aTargetRooms[4];
            break;
        case "petalburg":
            this.aForcedRoom = this.aTargetRooms[5];
            break;
        case "petalburgWoods":
            this.aForcedRoom = this.aTargetRooms[6];
            break;
        case "rustboro":
            this.aForcedRoom = this.aTargetRooms[7];
            break;
        default:
            this.aForcedRoom = null;
    }
}

public void resetForcedRoom(){
    this.aForcedRoom = null;
}

```

```

this.aIsTest = true;

String vFileName = pCommand.getSecondWord();
vFileName = vFileName + ".txt";

InputStream vIS = this.getClass().getClassLoader().getResourceAsStream(vFileName);

if (vIS == null) {
    System.out.println("File not found: " + vFileName);
    return;
}

Scanner vSC = new Scanner(vIS);

while (vSC.hasNextLine()) {
    String vLigne = vSC.nextLine();
    this.interpretCommand(vLigne);
}

vSC.close();
this.aIsTest = false;

private void alea ( final Command pCommand) {
    if (this.aPlayer.getCurrentRoom().isTeleporterRoom()){
        TransporterRoom vTransporterRoom = (TransporterRoom) this.aPlayer.getCurrentRoom();
        if(pCommand.hasSecondWord() == false){
            vTransporterRoom.resetForcedRoom();
        } else{
            String vRoom = pCommand.getSecondWord();
            vTransporterRoom.setForcedRoom(vRoom);
        }
    } else {
        this.aGui.println("you are not in the Transporter Room or in test mode");
    }
} // alea()

```

Bonus :

Me sentant limite par la limite de 20 mo du rendu, j'ai voulu créer un système permettant de télécharger des fichiers depuis un serveur distant, mr bureau ma recommande d'utiliser une classe java téléchargeant depuis le serveur esiee, ce que j'ai donc fais.

Note : Merci à Gemini qui m'a beaucoup aidé dans la recherche de ressources pour rendre cette classe possible et expliquer clairement les explications parfois compliquées des ressources ainsi qu'à mon oncle qui m'a donné la solution pour rendre l'écran de chargement réactif au nombre d'asset en cours de téléchargement.

Classe AssetManager : Cette classe gère la communication avec le serveur distant. Via la bibliothèque HttpURLConnection, elle parcourt un tableau d'images (aAssets) et les télécharge dans un dossier local Images/. Afin d'optimiser les lancements ultérieurs, la méthode allAssetsDownloaded() vérifie si les fichiers sont déjà présents localement et ne télécharge que les éléments manquants. Un système de sécurité a également été pensé : chaque fichier a droit à 3 tentatives (aMaxRetries) et si la connexion échoue (par exemple, sans Wi-Fi), le jeu bascule automatiquement en mode hors-ligne en affichant un message d'erreur, permettant au joueur de jouer avec les images manquantes plutôt que de faire planter l'application.

Classe LoadingScreen : Afin de rendre l'attente agréable et informative pour le joueur, une interface graphique basée sur Swing a été créée. Elle intègre une barre de progression (JProgressBar). C'est ici qu'intervient la logique de réactivité : la méthode setProgressBar() calcule le pourcentage d'avancement en temps réel en divisant le numéro du fichier en cours de téléchargement (getCurrentProgress()) par le nombre total de fichiers (getTotalAssets()) transmis par l'AssetManager.

Liaison dans la classe Game : L'initialisation de ce système se fait tout au début du jeu, dans le constructeur de la classe Game. Avant même de charger le moteur (GameEngine) ou l'interface principale (UserInterface), le programme instancie l'AssetManager. Si des fichiers manquent, l'écran de chargement est invoqué et la méthode startAndWait() met le lancement du jeu en attente jusqu'à ce que la récupération des données soit terminée, soit annulée par le mode hors-ligne.

```
public class Game {
    /** Swing-based view that shows text, images, and command controls. */
    private UserInterface aGui;
    /** Core game logic: rooms, parser, and command interpretation. */
    private GameEngine aEngine;
    /** Asset manager responsible for downloading missing files at startup. */
    private AssetManager aAssetManager;

    /**
     * Constructs the game by creating the engine and GUI, then linking them.
     * <p>
     * The GUI receives a reference to the engine; the engine is given the GUI
     * so it can print messages and update the display after construction.
     * </p>
     */
    public Game() {
        this.aAssetManager = new AssetManager();
        if (!aAssetManager.allAssetsDownloaded()) {
            LoadingScreen vLoading = new LoadingScreen(aAssetManager);
            aAssetManager.setLoadingScreen(vLoading);
            vLoading.startAndWait();
        }

        this.aEngine = new GameEngine();
        this.aGui = new UserInterface(this.aEngine);
        this.aEngine.setGUI(this.aGui);
    } // Game()
} // Game

public void downloadAssets() {
    int vFailCount = 0;

    for (int vI = 0; vI < aTotalAssets; vI++) {
        String vFileName = aAssets[vI];
        File vLocalFile = new File(aAssetsFolder + File.separator + vFileName);

        if (vLocalFile.exists()) {
            // already stored locally, skip
            this.aCurrentFile = vFileName;
            this.aCurrentProgress = vI + 1;
            continue;
        }

        boolean vSuccess = false;
        for (int vAttempt = 1; vAttempt <= aMaxRetries; vAttempt++) {
            vSuccess = this.downloadFile(vFileName, vLocalFile);
            if (vSuccess) {
                break;
            }
        }

        if (!vSuccess) {
            vFailCount++;
            System.out.println("Failed to download " + vFileName + " after " + aMaxRetries + " attempts.");
        }

        this.aCurrentFile = vFileName;
        this.aCurrentProgress = vI + 1;
    }

    if (vFailCount > 0) {
        this.aErrorMessage = vFailCount + " file(s) could not be downloaded. Running in offline mode." + "\n"
            + "Try restarting the game with Wi-Fi, or consider downloading the offline version from the website.";
        System.out.println(aErrorMessage);
    }
    this.aIsComplete = true;
} // downloadAssets()
```

```

/**
 * Downloads a single file from the remote server to the local path.
 *
 * @param pFileName the filename on the server (e.g. {@code "home.gif"})
 * @param pLocalFile the local destination file
 * @return {@code true} if the download succeeded; {@code false} otherwise
 */
private boolean downloadFile(final String pFileName, final File pLocalFile) {
    HttpURLConnection vConnection = null;
    try {
        // URL-encode the filename (handles spaces -> %20).
        String vEncodedName = URLEncoder.encode(pFileName, "UTF-8").replace("+", "%20");
        URL vUrl = new URL(aAssetsUrl + vEncodedName);

        vConnection = (HttpURLConnection) vUrl.openConnection();
        vConnection.setConnectTimeout(10000);
        vConnection.setReadTimeout(10000);
        vConnection.setRequestMethod("GET");

        int vResponseCode = vConnection.getResponseCode();
        if (vResponseCode != HttpURLConnection.HTTP_OK) {
            System.out.println("HTTP " + vResponseCode + " for: " + pFileName);
            return false;
        }

        try (InputStream vInput = vConnection.getInputStream();
             FileOutputStream vOutput = new FileOutputStream(pLocalFile)) {
            vInput.transferTo(vOutput);
        }

        return true;
    } catch (IOException vException) {
        System.out.println("Download error for " + pFileName + ": " + vException.getMessage());
        // Clean up any partial file.
        if (pLocalFile.exists()) {
            pLocalFile.delete();
        }
        return false;
    } finally {
        if (vConnection != null) {
            vConnection.disconnect();
        }
        if (this.aLoadingScreen != null) {
            this.aLoadingScreen.setProgressbar();
        }
    }
}
// downloadFile()

```

```

/**
 * Builds the loading screen UI: title, progress bar, and status labels.
 */
private void createGUI() {
    this.aFrame = new JFrame("Pokémon Delta Emerald - Loading");
    this.aFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.aFrame.setResizable(false);

    JPanel vPanel = new JPanel();
    vPanel.setLayout(new BorderLayout(10, 10));
    vPanel.setBorder(BorderFactory.createEmptyBorder(30, 40, 30, 40));
    vPanel.setBackground(new Color(30, 30, 40));

    // Title
    JLabel vTitle = new JLabel("Pokémon Delta Emerald", JLabel.CENTER);
    vTitle.setFont(new Font("SansSerif", Font.BOLD, 50));
    vTitle.setForeground(new Color(255, 215, 0));

    // Progress bar
    this.aProgressBar = new JProgressBar(0, 100);
    this.aProgressBar.setStringPainted(true);
    this.aProgressBar.setPreferredSize(new Dimension(400, 30));
    this.aProgressBar.setForeground(new Color(76, 175, 80));
    this.aProgressBar.setBackground(new Color(50, 50, 60));

    // Status label
    this.aStatusLabel = new JLabel("Checking assets...", JLabel.CENTER);
    this.aStatusLabel.setFont(new Font("SansSerif", Font.PLAIN, 14));
    this.aStatusLabel.setForeground(new Color(200, 200, 210));

    // Count label
    this.aCountLabel = new JLabel(" ", JLabel.CENTER);
    this.aCountLabel.setFont(new Font("SansSerif", Font.PLAIN, 13));
    this.aCountLabel.setForeground(new Color(160, 160, 170));

    // Layout: south panel for labels
    JPanel vSouthPanel = new JPanel();
    vSouthPanel.setLayout(new BorderLayout(5, 5));
    vSouthPanel.setOpaque(false);
    vSouthPanel.add(this.aStatusLabel, BorderLayout.NORTH);
    vSouthPanel.add(this.aCountLabel, BorderLayout.SOUTH);

    vPanel.add(vTitle, BorderLayout.NORTH);
    vPanel.add(this.aProgressBar, BorderLayout.CENTER);
    vPanel.add(vSouthPanel, BorderLayout.SOUTH);

    this.aFrame.getContentPane().add(vPanel);
    this.aFrame.pack();
    this.aFrame.setLocationRelativeTo(null);
} // createGUI()

```

```

public void setProgressBar( )
{
    int vProgress = (int) (((double) aAssetManager.getCurrentProgress() / aAssetManager.getTotalAssets()) * 100);
    this.aProgressBar.setValue(vProgress);
}

```

```

/**
 * Shows the loading screen.
 * This method blocks until the download finishes (success or offline fallback).
 */
public void startAndWait() {
    this.aFrame.setVisible(true);
    aAssetManager.downloadAssets();
    while (aAssetManager.isComplete() == false) {
    }

    if (aAssetManager.getErrorMessage() != null) {
        aStatusLabel.setText(aAssetManager.getErrorMessage());
        aCountLabel.setText("Starting in offline mode...");
        try {
            Thread.sleep(3000);
        } catch (InterruptedException ignore) {
        }
        this.aFrame.setVisible(false);
        this.aFrame.dispose();
        return;
    }

    this.aProgressBar.setValue(100);
    aStatusLabel.setText("All assets downloaded! Starting game...");
    try {
        Thread.sleep(2000);
    } catch (InterruptedException ignore) {
    }
    this.aFrame.setVisible(false);
    this.aFrame.dispose();
}

```

Ajout d'un effet à la condition de victoire qui sauvegarde si le joueur a gagné, et met en pause le timer de défaite.

```
public boolean hasWon() {
    return this.aHasWon;
}

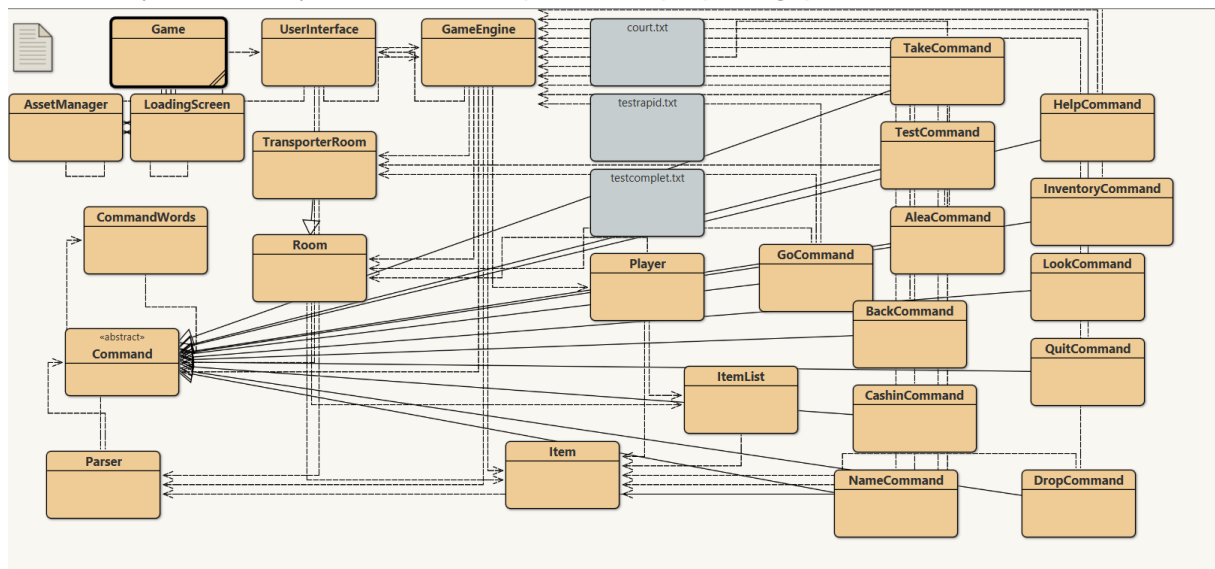
public void setHasWon(final boolean pHasWon) {
    this.aHasWon = pHasWon;
}

if (this.aPlayer.hasWon()) {
    return;
}

if (this.aPlayer.getItem("delta-orb") != null && this.aPlayer.getCurrentRoom().isWinningRoom()) {
    this.aGui.println("\n" + "Congratulations! You just won! Youve stopped rayquazza from destroying the wo
    this.aPlayer.setHasWon(true);
}
```

Exercice 7.47:

Pour rendre le code plus modulaire, j'ai procédé à une refabrication complète du système de commandes. La méthode interpretCommand a été allégée grâce à l'implémentation du design de Commande. La classe Command est devenu une classe abstraite, et chaque commande spécifique du jeu (GoCommand, TakeCommand, CashinCommand, etc.) est désormais une extension (sous-classe) de celle-ci, encapsulant sa propre logique d'exécution.




```

public class CashinCommand extends Command
{
    public CashinCommand()
    {
    }

    public boolean execute(GameEngine gameEngine)
    {
        if (!hasSecondWord()) {
            gameEngine.getGui().println("You need something to cash in!");
            return false;
        }
        String vItemName = getSecondWord();
        HashMap<String, Integer> vCashableItem = new HashMap<String, Integer>();
        vCashableItem.put("grant", 75);
        vCashableItem.put("wallet", 25);

        if (vCashableItem.containsKey(vItemName)) {
            Item vItem = gameEngine.getPlayer().getItem(vItemName);
            if (vItem != null) {
                int vReward = vCashableItem.get(vItemName);
                gameEngine.getPlayer().setMoney(gameEngine.getPlayer().getMoney() + vReward);
                gameEngine.getPlayer().removeItem(vItemName);
                gameEngine.getGui().println("You have cashed in your " + vItemName + " and are now richer!");
                gameEngine.getGui().println("Your current money is: " + gameEngine.getPlayer().getMoney());
            } else {
                gameEngine.getGui().println("You don't have a " + vItemName + " to cash in.");
            }
        } else {
            gameEngine.getGui().println("You can't cash that in.");
        }
        return false;
    }
}

public Command getCommand(final String vInput) {
    String vInputLine = vInput;
    String vWord1;
    String vWord2;

    StringTokenizer tokenizer = new StringTokenizer(vInputLine);

    if (tokenizer.hasMoreTokens()) {
        vWord1 = tokenizer.nextToken();
    } else {
        vWord1 = null;
    }

    if (tokenizer.hasMoreTokens()) {
        vWord2 = tokenizer.nextToken();
    } else {
        vWord2 = null;
    }

    if (aCommand.isCommand(vWord1) == false) {
        return null;
    }

    Command command = aCommand.get(vWord1);
    command.setSecondWord(null);
    if (command != null) {
        command.setSecondWord(vWord2);
    }

    return command;
} // getCommand()

/**
 * Indicates whether the parser recognized the command word.
 *
 * * @return {@code true} if {@link #getCommandWord()} is {@code null}
 */
public boolean isUnknown() {
    return this.aCommandWord == null;
} // isUnknown()

public abstract boolean execute(GameEngine gameEngine);

```

```

public boolean isTest() {
    return this.aIsTest;
}

public void setTest(boolean pIsTest) {
    this.aIsTest = pIsTest;
}

public Player getPlayer() {
    return this.aPlayer;
}

public UserInterface getGui() {
    return this.aGui;
}

/**
 * Creates a CommandWords object containing all valid commands.
 */
public CommandWords() {
    aCommands = new HashMap<String, Command>();
    aCommands.put("go", new GoCommand());
    aCommands.put("help", new HelpCommand());
    aCommands.put("quit", new QuitCommand());
    aCommands.put("look", new LookCommand());
    aCommands.put("cashin", new CashinCommand());
    aCommands.put("back", new BackCommand());
    aCommands.put("test", new TestCommand());
    aCommands.put("alea", new AleaCommand());
    aCommands.put("name", new NameCommand());
    aCommands.put("take", new TakeCommand());
    aCommands.put("drop", new DropCommand());
    aCommands.put("inventory", new InventoryCommand());
}

/**
 * Given a command word, find and return the matching command object.
 * Return null if there is no command with this name.
 */
public Command get(final String pWord) {
    return aCommands.get(pWord);
}

/**
 * Tests membership of {@code pString} in the internal command table
 * (case-sensitive).
 *
 * @param pString candidate verb, usually lower-case
 * @return {@code true} if {@code pString} is a known command word
 */
public boolean isCommand(final String pString) {
    return aCommands.containsKey(pString);
} // isCommand()

```

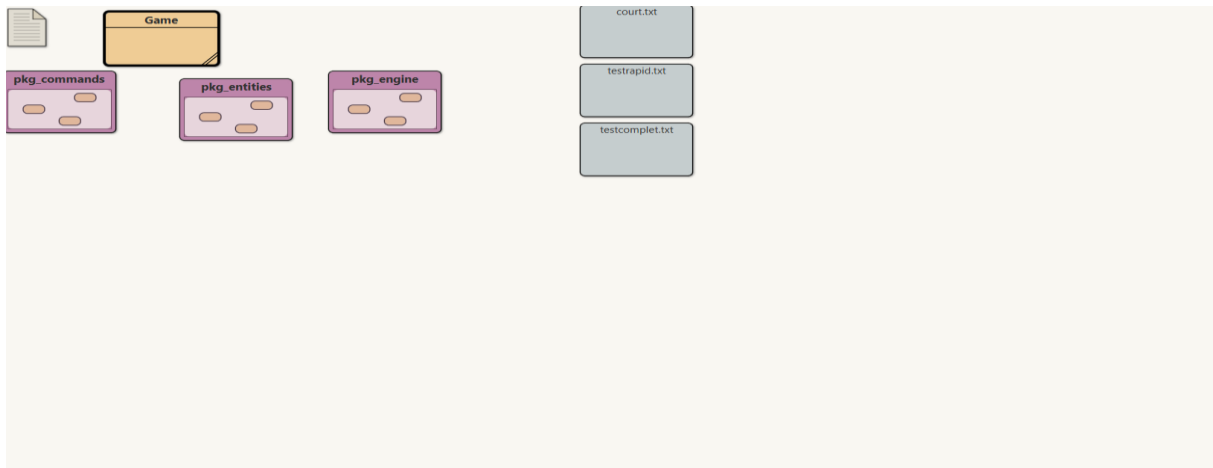
Exercice 7.47.1:

Afin d'améliorer la lisibilité et l'organisation de l'architecture grandissante du projet, j'ai réparti les classes dans différents sous-dossiers (packages). L'application est maintenant divisée logiquement entre pkg_engine (pour le moteur, l'IHM et l'AssetManager), pkg_commands (pour les objets commandes) et pkg_entities (pour le joueur, les pièces, les objets et les dresseurs).

```

import pkg_engine.AssetManager;
import pkg_engine.GameEngine;
import pkg_engine.LoadingScreen;
import pkg_engine.UserInterface;

```



Exercice 7.48:

J'ai ajouté des dresseurs Pokémon dans le jeu en créant la classe `Trainer`. Cette entité stocke leur phrase d'accroche (`aDialog`) ainsi que leur équipe Pokémon, gérée dynamiquement via une `HashMap<String, Integer>` (`aTeam`). Sur le même modèle que l'inventaire des objets, j'ai créé une classe `TrainerList` pour gérer la collection de dresseurs présents dans une pièce. Enfin, j'ai initialisé plusieurs dresseurs statiques (Javier, Maria, Sophie, Rayquaza) dans le `GameEngine` en les assignant à des pièces spécifiques via la méthode `addTrainer`

```
/** Narrative name or sentence shown in room listings. */
private String aDialog;
/** Numeric attribute (treated as price in user-facing strings). */
private HashMap<String, Integer> aTeam;
```

```
public Trainer(final String pDialog) {
    this.aDialog = pDialog;
    this.aTeam = new HashMap<>();
}
```

```
/**
 * Returns the numeric team size associated with this trainer.
 *
 * @return the configured team size
 */
public int getTeamSize() {
    return this.aTeam.size();
} // getTeamSize()
```

```
/**
 * Adds or replaces a pokemon under the given key.
 *
 * @param pPokemon the pokemon name used for later retrieval
 * @param pPosition the numeric team size to store
 */
public void setPokemon(final String pPokemon, final Integer pPosition) {
    this.aTeam.put(pPokemon, pPosition);
} // setPokemon()
```

```
/**
 * Builds a short multi-line summary combining description and team size.
 *
 * @return two-line English text suitable for room descriptions
 */
public String getTrainerDescription() {
    return this.aDialog + '\n' + "Team size: " + this.aTeam.size();
}
```

```

public Trainer getTrainer(final String pName) {
    return this.aTrainers.get(pName);
}

```

```

/**
 * Adds or replaces a trainer under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pTrainer the {@link Trainer} to store
 */
public void addTrainer(final String pName, final Trainer pTrainer) {
    this.aTrainers.put(pName, pTrainer);
}

```

```

/**
 * Removes the trainer associated with the given key.
 *
 * @param pName the lookup key
 */
public void removeTrainer(final String pName) {
    this.aTrainers.remove(pName);
}

```

```

/**
 * Checks whether this trainer list contains no elements.
 *
 * @return {@code true} when no trainers are stored
 */
public boolean isEmpty() {
    return this.aTrainers.isEmpty();
}

```

```

/**
 * Builds a multi-line string with each trainer's description,
 * or a default notice when the list is empty.
 *
 * @return formatted trainer descriptions
 */
public String getTrainersDescription() {
    if (this.aTrainers.isEmpty()) {
        return "There are no trainers here!";
    }
    String vResult = "";
    for (String vName : this.aTrainers.keySet()) {
        vResult += "There is " + vName + ", " + this.aTrainers.get(vName).getTrainerDescription() + "\n";
    }
    return vResult;
}

```

```

/**
 * Adds or replaces a trainer under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pTrainer the {@link Trainer} to store
 */
public void addTrainer(final String pName, final Trainer pTrainer) {
    this.aTrainers.addTrainer(pName, pTrainer);
}

```

```

Trainer Javier, Maria, Sophie, Rayquaza;
Javier = new Trainer("a young Pokémon trainer from Littleroot Town.");
Maria = new Trainer("an experienced Pokémon trainer from Rustboro City.");
Sophie = new Trainer("a skilled Pokémon trainer who lives in Petalburg Woods.");
Rayquaza = new Trainer("the legendary Rayquaza, who protects the skies.");

route101.addTrainer("Javier", Javier);
route102.addTrainer("Maria", Maria);
petalburgWoods.addTrainer("Sophie", Sophie);
skyPillar.addTrainer("Rayquaza", Rayquaza);

Javier.setPokemon("Poochyena", 1);
Javier.setPokemon("Rattata", 2);
Maria.setPokemon("Blastoise", 1);
Sophie.setPokemon("Dragonite", 1);
Rayquaza.setPokemon("Rayquaza", 1);

```

Exercice 7.49:

Pour rendre le monde plus vivant, j'ai créé la classe `MovingTrainer`, qui hérite de la classe `Trainer`. Ces dresseurs enregistrent leur pièce de départ et disposent d'une méthode `tryMove()`. Cette méthode leur donne une chance sur six de se déplacer vers une sortie aléatoire adjacente, en excluant les `TransporterRoom` pour éviter des comportements imprévisibles sur la carte. J'ai optimisé le stockage dans le `GameEngine` en ajoutant une `ArrayList` spécifique pour ces dresseurs mobiles, ce qui permet d'appeler leur déplacement automatiquement à chaque action de mouvement du joueur (lors de l'utilisation de `go` ou `back`).

```
public Trainer(final String pName, final String pDialog) {
    this.aName = pName;
    this.aDialog = pDialog;
    this.aTeam = new HashMap<>();
}

public String getName() {
    return this.aName;
}

/**
 * Returns the numeric team size associated with this trainer.
 *
 * @return the configured team size
 */
public int getTeamSize() {
    return this.aTeam.size();
} // getTeamSize()

/**
 * Adds or replaces a pokemon under the given key.
 *
 * @param pPokemon the pokemon name used for later retrieval
 * @param pPosition the numeric team size to store
 */
public void setPokemon(final String pPokemon, final Integer pPosition) {
    this.aTeam.put(pPokemon, pPosition);
} // setPokemon()

/**
 * Builds a short multi-line summary combining description and team size.
 *
 * @return two-line English text suitable for room descriptions
 */
public String getTrainerDescription() {
    return this.aDialog + '\n' + "Team size: " + this.aTeam.size();
}
```

```

public TrainerList() {
    this.aTrainers = new HashMap<String, Trainer>();
} // TrainerList()

/**
 * Adds or replaces a trainer under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pTrainer the {@link Trainer} to store
 */
public void addTrainer(final String pName, final Trainer pTrainer) {
    this.aTrainers.put(pName, pTrainer);
}

/**
 * Removes the trainer associated with the given key.
 *
 * @param pName the lookup key
 */
public void removeTrainer(final String pName) {
    this.aTrainers.remove(pName);
}

/**
 * Checks whether this trainer list contains no elements.
 *
 * @return {@code true} when no trainers are stored
 */
public boolean isEmpty() {
    return this.aTrainers.isEmpty();
}

/**
 * Builds a multi-line string with each trainer's description,
 * or a default notice when the list is empty.
 *
 * @return formatted trainer descriptions
 */
public String getTrainersDescription() {
    if (this.aTrainers.isEmpty()) {
        return "There are no trainers here!";
    }
    String vResult = "";
    for (String vName : this.aTrainers.keySet()) {
        vResult += "There is " + vName + ", " + this.aTrainers.get(vName).getTrainerDescription() + "\n";
    }
    return vResult;
}

public void addTrainer(final Trainer pTrainer) {
    this.aTrainers.addTrainer(pTrainer.getName(), pTrainer);
}

/**
 * Removes the trainer associated with the given key.
 *
 * @param pName the lookup key
 */
public void removeTrainer(final String pName) {
    this.aTrainers.removeTrainer(pName);
}

```

```

this.aMovingTrainers = new ArrayList<MovingTrainer>();
Trainer Rayquaza;
MovingTrainer Javier, Zucko, Ichigo;
Javier = new MovingTrainer("Javier", "a young Pokémon trainer from Littleroot Town.", route101);
this.aMovingTrainers.add(Javier);
Zucko = new MovingTrainer("Zucko", "an experienced Pokémon trainer from Rustboro City.", route102);
this.aMovingTrainers.add(Zucko);
Ichigo = new MovingTrainer("Ichigo", "a skilled Pokémon trainer who lives in Petalburg Woods.", petalburgWoods);
this.aMovingTrainers.add(Ichigo);
Rayquaza = new Trainer("Rayquaza", "the legendary Rayquaza, who protects the skies.");

route101.addTrainer(Javier);
route102.addTrainer(Zucko);
petalburgWoods.addTrainer(Ichigo);
skyPillar.addTrainer(Rayquaza);

Javier.setPokemon("Poochyena", 1);
Javier.setPokemon("Rattata", 2);
Zucko.setPokemon("Blastoise", 1);
Zucko.setPokemon("Gyarados", 2);
Zucko.setPokemon("Metagross", 3);
Ichigo.setPokemon("Dragonite", 1);
Ichigo.setPokemon("Salamence", 2);
Ichigo.setPokemon("Garchomp", 3);
Ichigo.setPokemon("Hydreigon", 4);
Rayquaza.setPokemon("Rayquaza", 1);

```

```

for (MovingTrainer vMovingTrainer : gameEngine.getMovingTrainers()) {
    vMovingTrainer.tryMove();
}

```

Exercice 7.53/54 :

Pour s'affranchir totalement de l'environnement de développement BlueJ, j'ai ajouté la méthode standard public static void main(String[] args) au sein de la classe Game. Cette méthode instancie un nouvel objet Game et sert de point d'entrée officiel pour lancer l'application en ligne de commande de manière autonome.

```

/**
 * Entry point when running from command line.
 * Creates a Game object to start the application.
 *
 * @param args command-line arguments (not used)
 */
public static void main(String[] args) {
    new Game();
} // main()

```

● PS C:\Users\j\ouan\Documents\GitHub\Barnabe-Jouanard\IPO\Zuul> java Game

Exercice 7.58/58.1.2 :

L'aboutissement du projet a été la création de l'archive exécutable .jar. J'ai compilé le projet via le terminal Windows en incluant toutes les classes des nouveaux packages ainsi que le dossier des ressources graphiques (jar cfe zuul.jar Game ...). Le jeu peut désormais être exécuté d'un simple java -jar Zuul.jar. Enfin, j'ai mis le jeu à disposition sur ma page web étudiante, en proposant au téléchargement la version standard, la version hors-ligne (pour pallier les problèmes de connexion), ainsi que les codes sources du projet (Note : Merci à Gemini pour le code HTML de la page web).

● PS C:\Users\j\ouan\Documents\GitHub\Barnabe-Jouanard\IPO\Zuul> jar cfe Zuul.jar Game *.class pkg_engine/*.class pkg_entities/*.class pkg_commands/*.class Images/
 ● PS C:\Users\j\ouan\Documents\GitHub\Barnabe-Jouanard\IPO\Zuul> java -jar Zuul.jar



Exercice 7.61/62 :

J'ai opté pour la première solution, à savoir sauvegarder l'état complet du jeu au lieu d'enregistrer les commandes exécutées. Ce choix était indispensable pour pouvoir restaurer la partie de manière fiable, car le jeu comporte des éléments aléatoires (notamment les déplacements des MovingTrainer) qui empêcheraient de retrouver l'état exact en rejouant simplement les commandes. Pour la sauvegarde j'ai implémenté la commande save qui génère un fichier .sav. L'état y est écrit en utilisant des identifications (par exemple : PLAYER_NAME:, CURRENT_ROOM:, INVENTORY:, BACK_STACK:). Techniquement, pour rendre cela possible, j'ai ajouté un identifiant aRoomId à la classe Room (référéncé via une HashMap dans GameEngine pour un accès en et un attribut alsDefeated dans Trainer. Pour le rechargement, j'ai créé la commande load. Lors de son appel, le jeu commence par nettoyer la session en cours (vidage de l'inventaire actuel et de la pile de retour back). Le fichier de sauvegarde est ensuite lu et restaure l'historique de navigation, le solde, le contenu de chaque pièce (items/trainers) et de replacer correctement les movingTrainer.

(Merci a Gemini pour le plan d'implémentation de cet exercice avec la checklist qui m'ont énormément aidé)


```

public boolean execute(GameEngine gameEngine)
{
    if (!hasSecondWord()) {
        gameEngine.getGui().println("Save to what file? Usage: save <filename>");
        return false;
    }

    String vFileName = getSecondWord() + ".sav";

    try (FileWriter vWriter = new FileWriter(vFileName)) {
        // Player name
        vWriter.write("PLAYER_NAME:" + gameEngine.getPlayer().getName() + "\n");

        // Current room
        vWriter.write("CURRENT_ROOM:" + gameEngine.getPlayer().getCurrentRoom().getRoomId() + "\n");

        // Money
        vWriter.write("MONEY:" + gameEngine.getPlayer().getMoney() + "\n");

        // Inventory
        vWriter.write("INVENTORY:");
        for (String vItemName : gameEngine.getPlayer().getItems().keySet()) {
            Item vItem = gameEngine.getPlayer().getItem(vItemName);
            vWriter.write(vItemName + "-" + vItem.getItemPrice() + ";");
        }
        vWriter.write("\n");

        // Back stack
        vWriter.write("BACK_STACK:");
        for (pkg_entities.Room vRoom : gameEngine.getPlayer().getPreviousRooms()) {
            vWriter.write(vRoom.getRoomId() + ",");
        }
        vWriter.write("\n");

        // Moving trainers
        for (pkg_entities.MovingTrainer vTrainer : gameEngine.getMovingTrainers()) {
            vWriter.write("MTRAINER:" + vTrainer.getName() + "-" + vTrainer.isDefeated() + "-" + vTrainer.getCurrentRoomId() + ";");
        }

        // Room states
        for (String vRoomId : gameEngine.getAllRoomIds()) {
            pkg_entities.Room vRoom = gameEngine.getRoomById(vRoomId);
            vWriter.write("ROOM:" + vRoomId + "-");
            for (String vItemName : vRoom.getItems().keySet()) {
                vWriter.write(vItemName + "=" + vRoom.getItems().get(vItemName).getItemPrice() + ";");
            }
            vWriter.write("\n");
        }
    }

    gameEngine.getGui().println("Game saved to " + vFileName);
    gameEngine.getGui().println("Goodbye!");

    return true;
} catch (IOException e) {
    gameEngine.getGui().println("Error saving game: " + e.getMessage());
    return false;
}

public boolean isDefeated() {
    return this.aIsDefeated;
}

public void setDefeated(boolean pDefeated) {
    this.aIsDefeated = pDefeated;
}

public java.util.HashMap<String, Item> getItems() {
    return this.aItems.getAllItems();
}

public java.util.Stack<Room> getPreviousRooms() {
    return this.aPreviousRooms;
}

public HashMap<String, Item> getAllItems() {
    return this.aItems;
}

```

```

public void setRoomId(String pId) {
    this.aRoomId = pId;
}

```

```

public String getRoomId() {
    return this.aRoomId;
}

```

```

house = new Room("in your house in Littleroot Town.", "home.gif", "home_map.jpeg");
house.setRoomId("house");
this.aRooms.put("house", house);
vTeleportableRooms.add(house);
littleroot = new Room("in Littleroot Town, your home town.", "littleroot town.gif", "littleroot_map.jpeg");
littleroot.setRoomId("littleroot");
this.aRooms.put("littleroot", littleroot);
vTeleportableRooms.add(littleroot);
route101 = new Room("on Route 101; wild Pokémon might appear in the tall grass.", "route 101.gif", "route101_map.jpeg");
route101.setRoomId("route101");
this.aRooms.put("route101", route101);
vTeleportableRooms.add(route101);
oldale = new Room("in Oldale Town, a small junction with a Pokémon Center.", "oldale town.gif", "oldale town_map.jpeg");
oldale.setRoomId("oldale");
this.aRooms.put("oldale", oldale);
vTeleportableRooms.add(oldale);
route102 = new Room("on Route 102; wild Pokémon might appear in the tall grass.", "route 102.gif", "route102_map.jpeg");
route102.setRoomId("route102");
this.aRooms.put("route102", route102);
vTeleportableRooms.add(route102);
petalburg = new Room("in Petalburg City, where your father is the Gym Leader.", "petalburg city.gif", "petalburg city_map.jpeg");
petalburg.setRoomId("petalburg");
this.aRooms.put("petalburg", petalburg);
vTeleportableRooms.add(petalburg);
petalburgWoods = new Room("in Petalburg Woods.", "petalburg woods.gif", "petalburg woods_map.jpeg");
petalburgWoods.setRoomId("petalburgWoods");
this.aRooms.put("petalburgWoods", petalburgWoods);
vTeleportableRooms.add(petalburgWoods);
rustboro = new Room("in Rustboro City, where the Devon Corporation is located.", "rustboro city.gif", "rustboro city_map.jpeg");
rustboro.setRoomId("rustboro");
this.aRooms.put("rustboro", rustboro);
vTeleportableRooms.add(rustboro);
skyPillar = new Room("on the Sky Pillar, Rayquaza's home.", "sky pillar.gif", "sky pillar_map.jpeg");
skyPillar.setRoomId("skyPillar");
this.aRooms.put("skyPillar", skyPillar);

```

```

if (!hasSecondWord()) {
    gameEngine.getGui().println("Load from what file? Usage: load <filename>");
    return false;
}

```

```
String vFileName = getSecondWord() + ".sav";
```

```

try (BufferedReader vReader = new BufferedReader(new FileReader(vFileName))) {
    String vLine;

    // Clear player inventory first
    for (String vItemName : new java.util.ArrayList<>(gameEngine.getPlayer().getItems().keySet())) {
        gameEngine.getPlayer().removeItem(vItemName);
    }

    // Clear back stack
    gameEngine.getPlayer().getPreviousRooms().clear();

    while ((vLine = vReader.readLine()) != null) {
        if (vLine.startsWith("PLAYER_NAME:")) {
            gameEngine.getPlayer().setName(vLine.substring(12));
        } else if (vLine.startsWith("CURRENT_ROOM:")) {
            Room vRoom = gameEngine.getRoomById(vLine.substring(13));
            if (vRoom != null) {
                gameEngine.getPlayer().setCurrentRoom(vRoom);
            }
        } else if (vLine.startsWith("MONEY:")) {
            gameEngine.getPlayer().setMoney(Integer.parseInt(vLine.substring(6)));
        } else if (vLine.startsWith("INVENTORY:")) {
            String vItems = vLine.substring(10);
            if (!vItems.isEmpty()) {
                for (String vPair : vItems.split(";")) {
                    if (!vPair.isEmpty()) {
                        String[] vParts = vPair.split("-");
                        if (vParts.length == 2) {
                            gameEngine.getPlayer().addItem(vParts[0], new Item(vParts[0], Integer.parseInt(vParts[1])));
                        }
                    }
                }
            }
        }
    }
}

```

```

    } else if (vLine.startsWith("BACK_STACK:")) {
        String vRooms = vLine.substring(11);
        if (!vRooms.isEmpty()) {
            for (String vRoomId : vRooms.split(",")) {
                if (!vRoomId.isEmpty()) {
                    Room vRoom = gameEngine.getRoomById(vRoomId);
                    if (vRoom != null) {
                        gameEngine.getPlayer().getPreviousRooms().push(vRoom);
                    }
                }
            }
        }
    }

    } else if (vLine.startsWith("MTRAINER:")) {
        String vData = vLine.substring(9);
        String[] vParts = vData.split("-");
        if (vParts.length == 3) {
            String vName = vParts[0];
            boolean vDefeated = Boolean.parseBoolean(vParts[1]);
            String vRoomId = vParts[2];

            for (pkg_entities.MovingTrainer vTrainer : gameEngine.getMovingTrainers()) {
                if (vTrainer.getName().equals(vName)) {
                    vTrainer.setDefeated(vDefeated);

                    Room vOldRoom = vTrainer.getCurrentRoom();
                    Room vNewRoom = gameEngine.getRoomById(vRoomId);
                    if (vOldRoom != null && vNewRoom != null && !vOldRoom.equals(vNewRoom)) {
                        vOldRoom.removeTrainer(vName);
                        vNewRoom.addTrainer(vTrainer);
                        vTrainer.setCurrentRoom(vNewRoom);
                    }
                    break;
                }
            }
        }
    }
}

```

```

    } else if (vLine.startsWith("ROOM:")) {
        String vData = vLine.substring(5);
        String[] vParts = vData.split("-");
        if (vParts.length == 2) {
            String vRoomId = vParts[0];
            String vItemsData = vParts[1];

            Room vRoom = gameEngine.getRoomById(vRoomId);
            if (vRoom != null) {
                // Clear existing items
                for (String vItemName : new java.util.ArrayList<>(vRoom.getItems().keySet())) {
                    vRoom.removeItem(vItemName);
                }

                // Add saved items
                if (!vItemsData.isEmpty()) {
                    for (String vItemPair : vItemsData.split(";")) {
                        if (!vItemPair.isEmpty()) {
                            String[] vItemParts = vItemPair.split("=");
                            if (vItemParts.length == 2) {
                                vRoom.addItem(vItemParts[0], new Item(vItemParts[0], Integer.parseInt(vItemParts[1])));
                            }
                        }
                    }
                }
            }
        }
    }

    gameEngine.getGui().println("Game loaded from " + vFileName);
    gameEngine.getGui().println(gameEngine.getPlayer().getCurrentRoom().getLongDescription());
    if (gameEngine.getPlayer().getCurrentRoom().getImageName() != null) {
        gameEngine.getGui().showImage(gameEngine.getPlayer().getCurrentRoom().getImageName());
        if (gameEngine.getPlayer().getItem("map") != null) {
            gameEngine.getGui().showMap(gameEngine.getPlayer().getCurrentRoom().getMapImageName());
        }
    }

    return false;
} catch (IOException e) {
    gameEngine.getGui().println("Error loading game: " + e.getMessage());
    return false;
} catch (NumberFormatException e) {
    gameEngine.getGui().println("Save file corrupted!");
    return false;
}
}

```

Exercice Bonus Final :

Ajout du système de combat permettant de battre Rayquaza et rendre la condition has won dépendante de cette victoire, création de la classe battle command qui lance un combat, combat qui est gère par la classe battle manager et comprend tous les mécaniques du combat telles que les changements, et autre, la classe attack, qui définit une attaque et renvoie les dégâts inflige avec une chance de rate l'attaque a cause de la précision, déclaration dans gamengine des équipes et attaques et dans user interface, création d'un gui secondaire qui montre les attaques des pokémons a utilisé et permet de fuir.

Ajout de la musique, pour ajouter un peu d'ambiance, j'ai utilisé la méthode trouve dans plus de techniques, et Gemini ma explique que pour pouvoir compile et garde l'exécution en java Main, il fallait extraire le jar a la racine du dossier, ce que j'ai donc fait pour plus de simplicité malgré que ce soit un peu brouillon.

```
private boolean aIsTurn;
public BattleCommand()
{
    this.aIsTurn = true;
}

public boolean execute(GameEngine gameEngine)
{
    if (!hasSecondWord()) {
        // List trainers in current room
        gameEngine.getGui().println("Trainers in this room:");
        java.util.ArrayList<String> vTrainerNames = gameEngine.getPlayer().getCurrentRoom().getTrainerNames();

        if (vTrainerNames.isEmpty()) {
            gameEngine.getGui().println("No trainers here!");
        } else {
            for (String vName : vTrainerNames) {
                gameEngine.getGui().println("- " + vName);
            }
            gameEngine.getGui().println("\nUse: battle <trainer_name>");
        }
        return false;
    }

    String vTrainerName = getSecondWord();

    // Check if trainer is in current room
    Room vCurrentRoom = gameEngine.getPlayer().getCurrentRoom();
    java.util.HashMap<String, Trainer> vTrainers = vCurrentRoom.getAllTrainers();
    Trainer vTrainer = vTrainers.get(vTrainerName);

    if (vTrainer == null) {
        gameEngine.getGui().println("There is no trainer by that name here!");
        return false;
    }

    if (vTrainer.isDefeated()) {
        gameEngine.getGui().println(vTrainerName + " has already been defeated!");
        return false;
    }

    if (vTrainerName == "Rayquaza" && gameEngine.getPlayer().getItem("delta-orb") == null) {
        gameEngine.getGui().println("You need the delta orb to battle Rayquaza! Find it and come back!");
        return false;
    }

    gameEngine.getBattleManager().startBattle(vTrainer);

    return false;
}

public boolean isTurn() {
    return this.aIsTurn;
}
```

```

private GameEngine aEngine;
private UserInterface aGui;
private Trainer aOpponent;
private Pokemon aPlayerPokemon;
private Pokemon aOpponentPokemon;
private int aPlayerCurrentHP;
private int aOpponentCurrentHP;
private boolean aBattleActive;
private int aPlayerPokemonLeft;
private int aOpponentPokemonLeft;
private int aPlayerPokemonIndex;
private int aOpponentPokemonIndex;

```

```

public BattleManager(GameEngine pEngine, UserInterface pGui) {
    this.aEngine = pEngine;
    this.aGui = pGui;
    this.aBattleActive = false;
}

```

```

/**
 * Starts a battle with the given trainer.
 */
public void startBattle(Trainer pOpponent) {
    this.aOpponent = pOpponent;

    // In test mode, auto-win the battle
    if (this.aEngine.isTest()) {
        this.aGui.println("\n[TEST] Auto-won battle against " + pOpponent.getName() + "!");
        pOpponent.setDefeated(true);
        if (pOpponent.getName().equals("Rayquaza")) {
            this.aEngine.getPlayer().setHasWon(true);
            this.aGui.println("Congratulations! You defeated Rayquaza and saved Hoenn!");
        }
        return;
    }

    this.aPlayerPokemonIndex = 1;
    this.aOpponentPokemonIndex = 1;
    this.aPlayerPokemon = this.aEngine.getPlayer().getPokemon(this.aPlayerPokemonIndex);
    this.aOpponentPokemon = pOpponent.getPokemon(this.aOpponentPokemonIndex);

    if (this.aPlayerPokemon == null || this.aOpponentPokemon == null) {
        this.aGui.println("Error: Missing Pokemon!");
        return;
    }
}

```

```

    this.aPlayerCurrentHP = this.aPlayerPokemon.getHp();
    this.aOpponentCurrentHP = this.aOpponentPokemon.getHp();
    this.aPlayerPokemonLeft = this.aEngine.getPlayer().getTeamSize();
    this.aOpponentPokemonLeft = pOpponent.getTeamSize();
    this.aBattleActive = true;

    // Show trainer images first, buttons disabled
    this.aGui.startBattle();
    this.aGui.showPlayerPokemonImage("trainer_battle.png");
    this.aGui.showOpponentPokemonImage(pOpponent.getName() + "_battle.png");
    this.updateHP();
    this.setupAttackButtons();
    this.setButtonsEnabled(false);
    this.aGui.println("\n" + pOpponent.getName() + " wants to battle!");

    // Wait 2s, then switch to pokemon solo images and enable buttons
    new Thread(() -> {
        try { Thread.sleep(2000); } catch (InterruptedException e) {}
        javax.swing.SwingUtilities.invokeLater(() -> {
            this.aGui.showPlayerPokemonImage(this.aPlayerPokemon.getName() + " solo.png");
            this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " solo.png");
            this.setButtonsEnabled(true);
        });
    }).start();
}

private void updateHP() {
    this.aGui.setPlayerHP(this.aPlayerCurrentHP, this.aPlayerPokemon.getHp());
    this.aGui.setOpponentHP(this.aOpponentCurrentHP, this.aOpponentPokemon.getHp());
}

private void clearListeners(javax.swing.JButton pButton) {
    for (java.awt.event.ActionListener al : pButton.getActionListeners()) {
        pButton.removeActionListener(al);
    }
}

```

```

private void setupAttackButtons() {
    clearListeners(this.aGui.getAttack1Button());
    clearListeners(this.aGui.getAttack2Button());
    clearListeners(this.aGui.getAttack3Button());
    clearListeners(this.aGui.getAttack4Button());
    clearListeners(this.aGui.getRunButton());

    this.aGui.getAttack1Button().setEnabled(true);
    this.aGui.getAttack2Button().setEnabled(true);
    this.aGui.getAttack3Button().setEnabled(true);
    this.aGui.getAttack4Button().setEnabled(true);
    this.aGui.getRunButton().setEnabled(true);

    java.util.HashMap<String, Integer> vMoves = this.aPlayerPokemon.getMoves();
    java.util.ArrayList<String> vMoveNames = new java.util.ArrayList<>(vMoves.keySet());

    if (vMoveNames.size() > 0) {
        this.aGui.getAttack1Button().setText(vMoveNames.get(0));
        this.aGui.getAttack1Button().addActionListener(e -> this.playerTurn(vMoveNames.get(0)));
    }
    if (vMoveNames.size() > 1) {
        this.aGui.getAttack2Button().setText(vMoveNames.get(1));
        this.aGui.getAttack2Button().addActionListener(e -> this.playerTurn(vMoveNames.get(1)));
    } else {
        this.aGui.getAttack2Button().setEnabled(false);
    }
    if (vMoveNames.size() > 2) {
        this.aGui.getAttack3Button().setText(vMoveNames.get(2));
        this.aGui.getAttack3Button().addActionListener(e -> this.playerTurn(vMoveNames.get(2)));
    } else {
        this.aGui.getAttack3Button().setEnabled(false);
    }
    if (vMoveNames.size() > 3) {
        this.aGui.getAttack4Button().setText(vMoveNames.get(3));
        this.aGui.getAttack4Button().addActionListener(e -> this.playerTurn(vMoveNames.get(3)));
    } else {
        this.aGui.getAttack4Button().setEnabled(false);
    }

    this.aGui.getRunButton().addActionListener(e -> this.runAway());
}

```

```

private void playerTurn(String pMoveName) {
    if (!this.aBattleActive) return;

    this.setButtonsEnabled(false);

    java.util.HashMap<String, Integer> vPlayerMoves = this.aPlayerPokemon.getMoves();
    Integer vDamage = vPlayerMoves.get(pMoveName);

    if (vDamage == null) {
        this.setButtonsEnabled(true);
        return;
    }

    // Player attacks - show damage gifs
    this.aGui.println("\n" + this.aPlayerPokemon.getName() + " used " + pMoveName + "!");
    this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " damage.gif");
    this.aOpponentCurrentHP -= vDamage;
    if (this.aOpponentCurrentHP < 0) this.aOpponentCurrentHP = 0;
    this.updateHP();

    // Wait 2s for animation, then process result
    new Thread(() -> {
        try { Thread.sleep(2000); } catch (InterruptedException e) {}
        javax.swing.SwingUtilities.invokeLater(() -> {
            // Check if opponent fainted
            if (this.aOpponentCurrentHP <= 0) {
                this.aGui.println(this.aOpponentPokemon.getName() + " fainted!");
                this.aOpponentPokemonLeft--;

                if (this.aOpponentPokemonLeft <= 0) {
                    this.endBattle(true);
                } else {
                    this.aOpponentPokemonIndex++;
                    this.aOpponentPokemon = this.aOpponent.getPokemon(this.aOpponentPokemonIndex);
                    this.aOpponentCurrentHP = this.aOpponentPokemon.getHp();
                    this.aGui.println(this.aOpponent.getName() + " sent out " + this.aOpponentPokemon.getName() + "!");
                    this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " solo.png");
                    this.aGui.showPlayerPokemonImage(this.aPlayerPokemon.getName() + " solo.png");
                    this.updateHP();
                    this.delayThenEnable();
                }
            }
            return;
        });
    });

    // Opponent attacks back - show player damage gif
    java.util.HashMap<String, Integer> vOpponentMoves = this.aOpponentPokemon.getMoves();

```

```

        if (vOpponentMoves.size() > 0) {
            String vOpponentMoveName = vOpponentMoves.keySet().iterator().next();
            Integer vOpponentDamage = vOpponentMoves.get(vOpponentMoveName);

            this.aGui.println(this.aOpponentPokemon.getName() + " used " + vOpponentMoveName + "!");
            this.aGui.showPlayerPokemonImage(this.aPlayerPokemon.getName() + " damage.gif");
            this.aPlayerCurrentHP -= vOpponentDamage;
            if (this.aPlayerCurrentHP < 0) this.aPlayerCurrentHP = 0;
            this.updateHP();

            // Wait 2s for player damage animation
            new Thread(() -> {
                try { Thread.sleep(2000); } catch (InterruptedException e2) {}
                javax.swing.SwingUtilities.invokeLater(() -> {
                    if (this.aPlayerCurrentHP <= 0) {
                        this.aGui.println(this.aPlayerPokemon.getName() + " fainted!");
                        this.aPlayerPokemonLeft--;

                        if (this.aPlayerPokemonLeft <= 0) {
                            this.endBattle(false);
                        } else {
                            this.aPlayerPokemonIndex++;
                            this.aPlayerPokemon = this.aEngine.getPlayer().getPokemon(this.aPlayerPokemonIndex);
                            this.aPlayerCurrentHP = this.aPlayerPokemon.getHp();
                            this.aGui.println("Go, " + this.aPlayerPokemon.getName() + "!");
                            this.aGui.showPlayerPokemonImage(this.aPlayerPokemon.getName() + " solo.png");
                            this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " solo.png");
                            this.updateHP();
                            this.setupAttackButtons();
                            this.delayThenEnable();
                        }
                    } else {
                        // Both alive - back to solo images, enable buttons
                        this.aGui.showPlayerPokemonImage(this.aPlayerPokemon.getName() + " solo.png");
                        this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " solo.png");
                        this.setButtonsEnabled(true);
                    }
                });
            }).start();
        } else {
            // Opponent has no moves
            this.aGui.showOpponentPokemonImage(this.aOpponentPokemon.getName() + " solo.png");
            this.setButtonsEnabled(true);
        }
    });
}
}).start();

```

```

private String aName;
private int aDamage;
private int aPrecision;
/**
 * Constructeur d'objets de classe Attack
 */
public Attack(final String pName, final int pDamage, final int pPrecision)
{
    this.aName = pName;
    this.aDamage = pDamage;
    this.aPrecision = pPrecision;
}

```

```

public int getDamage() {
    Random random = new Random();
    int roll = random.nextInt(101);

    int vFactor;

    if (roll > aPrecision) {
        vFactor = 0;
    } else {
        vFactor = 1;
    }
    return aDamage * vFactor;
}

```

```

public String getName() {
    return aName;
}

```



```

Attack dragonPulse = new Attack("Dragon Pulse", 50, 100);
Attack ironHead = new Attack("Iron Head", 50, 100);
Attack hyperBeam = new Attack("Hyper Beam", 100, 75);
Attack extremeSpeed = new Attack("Extreme Speed", 50, 100);
Attack darkPulse = new Attack("Dark Pulse", 50, 100);
Attack waterShuriken = new Attack("Water Shuriken", 65, 100);

// Javier's team - weak, easy to beat
Pokemon Poochyena = new Pokemon("Poochyena", 80, new Attack[]{tackle, bite});
Pokemon Rattata = new Pokemon("Rattata", 60, new Attack[]{tackle, bite});

// Zucko's team - medium difficulty
Pokemon Blastoise = new Pokemon("Blastoise", 180, new Attack[]{waterGun, hydroPump, iceBeam, bite});
Pokemon Gyarados = new Pokemon("Gyarados", 200, new Attack[]{hydroPump, bite, dragonClaw});
Pokemon Metagross = new Pokemon("Metagross", 220, new Attack[]{ironHead, earthquake, dragonClaw});

// Ichigo's team - hard, all dragons
Pokemon Dragonite = new Pokemon("Dragonite", 250, new Attack[]{dragonClaw, outrage, fireBlast, extremeSpeed});
Pokemon Salamence = new Pokemon("Salamence", 270, new Attack[]{dragonClaw, outrage, flamethrower, earthquake});
Pokemon Garchomp = new Pokemon("Garchomp", 300, new Attack[]{dragonClaw, earthquake, outrage, fireBlast});
Pokemon Hydreigon = new Pokemon("Hydreigon", 320, new Attack[]{darkPulse, dragonPulse, fireBlast, outrage});

// Rayquaza - boss fight, massive HP pool, 4 devastating moves
Pokemon RayquazaPokemon = new Pokemon("Rayquaza", 900, new Attack[]{outrage, hyperBeam, extremeSpeed, dragonClaw});

// Player team - strong enough to win if you don't waste HP
Pokemon Charizard = new Pokemon("Charizard", 250, new Attack[]{flamethrower, fireBlast, dragonClaw, bite});
Pokemon Greninja = new Pokemon("Greninja", 200, new Attack[]{waterShuriken, hydroPump, iceBeam, darkPulse});
Pokemon Incineroar = new Pokemon("Incineroar", 280, new Attack[]{flamethrower, fireBlast, bite, earthquake});

Javier.setPokemon(1, Poochyena);
Javier.setPokemon(2, Rattata);
Zucko.setPokemon(1, Blastoise);
Zucko.setPokemon(2, Gyarados);
Zucko.setPokemon(3, Metagross);
Ichigo.setPokemon(1, Dragonite);
Ichigo.setPokemon(2, Salamence);
Ichigo.setPokemon(3, Garchomp);
Ichigo.setPokemon(4, Hydreigon);
Rayquaza.setPokemon(1, RayquazaPokemon);

this.aPlayer = new Player("Luke", house);

this.aPlayer.setPokemon(1, Charizard);
this.aPlayer.setPokemon(2, Greninja);
this.aPlayer.setPokemon(3, Incineroar);

this.aIsTest = false;

```

```

private void createBattlePanel() {
    this.aBattlePanel = new JPanel();
    this.aBattlePanel.setLayout(new BorderLayout());
    this.aBattlePanel.setBackground(new Color(30, 30, 40));

    // HP bars panel (top)
    JPanel vHPPanel = new JPanel();
    vHPPanel.setLayout(new GridLayout(2, 1, 10, 10));
    vHPPanel.setBackground(new Color(30, 30, 40));

    this.aOpponentHPBar = new JProgressBar(0, 100);
    this.aOpponentHPBar.setStringPainted(true);
    this.aOpponentHPBar.setString("Opponent: 100/100");
    this.aOpponentHPBar.setForeground(new Color(244, 67, 54));
    this.aOpponentHPBar.setValue(100);
    this.aOpponentHPBar.setPreferredSize(new Dimension(300, 40));

    this.aPlayerHPBar = new JProgressBar(0, 100);
    this.aPlayerHPBar.setStringPainted(true);
    this.aPlayerHPBar.setString("Your Pokemon: 100/100");
    this.aPlayerHPBar.setForeground(new Color(76, 175, 80));
    this.aPlayerHPBar.setValue(100);
    this.aPlayerHPBar.setPreferredSize(new Dimension(300, 40));

    vHPPanel.add(this.aOpponentHPBar);
    vHPPanel.add(this.aPlayerHPBar);

    // Pokemon images panel (center)
    JPanel vPokemonPanel = new JPanel();
    vPokemonPanel.setLayout(new GridLayout(1, 2, 20, 0));
    vPokemonPanel.setBackground(new Color(30, 30, 40));

    this.aPlayerPokemonImage = new JLabel();
    this.aPlayerPokemonImage.setHorizontalAlignment(JLabel.CENTER);
    this.aPlayerPokemonImage.setVerticalAlignment(JLabel.CENTER);

    this.aOpponentPokemonImage = new JLabel();
    this.aOpponentPokemonImage.setHorizontalAlignment(JLabel.CENTER);
    this.aOpponentPokemonImage.setVerticalAlignment(JLabel.CENTER);

    vPokemonPanel.add(this.aPlayerPokemonImage);
    vPokemonPanel.add(this.aOpponentPokemonImage);
}

```

```

// Pokemon images panel (center)
JPanel vPokemonPanel = new JPanel();
vPokemonPanel.setLayout(new GridLayout(1, 2, 20, 0));
vPokemonPanel.setBackground(new Color(30, 30, 40));

this.aPlayerPokemonImage = new JLabel();
this.aPlayerPokemonImage.setHorizontalAlignment(JLabel.CENTER);
this.aPlayerPokemonImage.setVerticalAlignment(JLabel.CENTER);

this.aOpponentPokemonImage = new JLabel();
this.aOpponentPokemonImage.setHorizontalAlignment(JLabel.CENTER);
this.aOpponentPokemonImage.setVerticalAlignment(JLabel.CENTER);

vPokemonPanel.add(this.aPlayerPokemonImage);
vPokemonPanel.add(this.aOpponentPokemonImage);

// Attack buttons panel (bottom)
JPanel vAttackPanel = new JPanel();
vAttackPanel.setLayout(new GridLayout(2, 3, 5, 5));
vAttackPanel.setPreferredSize(new Dimension(0, 120));

this.aAttack1Button = new JButton("Attack 1");
this.aAttack2Button = new JButton("Attack 2");
this.aAttack3Button = new JButton("Attack 3");
this.aAttack4Button = new JButton("Attack 4");
this.aRunButton = new JButton("Run Away");

vAttackPanel.add(this.aAttack1Button);
vAttackPanel.add(this.aAttack2Button);
vAttackPanel.add(this.aAttack3Button);
vAttackPanel.add(this.aAttack4Button);
vAttackPanel.add(new JLabel()); // Empty space
vAttackPanel.add(this.aRunButton);

this.aBattlePanel.add(vHPPanel, BorderLayout.NORTH);
this.aBattlePanel.add(vPokemonPanel, BorderLayout.CENTER);
this.aBattlePanel.add(vAttackPanel, BorderLayout.SOUTH);

this.aBattlePanel.setVisible(false);
} // createBattlePanel()

/**
 * Switches from exploration mode to battle mode.
 */
public void startBattle() {
    this.aMyFrame.getContentPane().remove(this.aExplorationPanel);
    this.aMyFrame.getContentPane().add(this.aBattlePanel, BorderLayout.CENTER);
    this.aBattlePanel.setVisible(true);
    this.aMyFrame.revalidate();
    this.aMyFrame.repaint();
} // startBattle()

```

```

/**
 * Switches from exploration mode to battle mode.
 */
public void startBattle() {
    this.aMyFrame.getContentPane().remove(this.aExplorationPanel);
    this.aMyFrame.getContentPane().add(this.aBattlePanel, BorderLayout.CENTER);
    this.aBattlePanel.setVisible(true);
    this.aMyFrame.revalidate();
    this.aMyFrame.repaint();
} // startBattle()

/**
 * Switches from battle mode back to exploration mode.
 */
public void endBattle() {
    this.aMyFrame.getContentPane().remove(this.aBattlePanel);
    this.aMyFrame.getContentPane().add(this.aExplorationPanel, BorderLayout.CENTER);
    this.aExplorationPanel.setVisible(true);
    this.aMyFrame.revalidate();
    this.aMyFrame.repaint();
} // endBattle()

/**
 * Shows player's Pokemon image (left side).
 * PNG files are scaled to 640x400, GIF files are displayed raw.
 * @param pImageName resource path for player pokemon
 */
public void showPlayerPokemonImage(final String pImageName) {
    String vImagePath = "Images/" + pImageName;
    URL vImageURL = this.getClass().getClassLoader().getResource(vImagePath);
    if (vImageURL == null) {
        System.out.println("Player pokemon image not found: " + vImagePath);
    } else {
        if (pImageName.endsWith(".png")) {
            ImageIcon vIcon = new ImageIcon(vImageURL);
            this.aPlayerPokemonImage.setIcon(new ImageIcon(vIcon.getImage().getScaledInstance(960, 904, java.awt.Image.SCALE_SMOOTH)));
        } else {
            ImageIcon vIcon = new ImageIcon(vImageURL);
            vIcon.getImage().flush();
            this.aPlayerPokemonImage.setIcon(vIcon);
        }
    }
} // showPlayerPokemonImage()

```

```

/**
 * Shows opponent's Pokemon image (right side).
 * PNG files are scaled to 960x904, GIF files are displayed raw with animation.
 * @param pImageName resource path for opponent pokemon
 */
public void showOpponentPokemonImage(final String pImageName) {
    String vImagePath = "Images/" + pImageName;
    URL vImageURL = this.getClass().getClassLoader().getResource(vImagePath);
    if (vImageURL == null) {
        System.out.println("Opponent pokemon image not found: " + vImagePath);
    } else {
        if (pImageName.endsWith(".png")) {
            ImageIcon vIcon = new ImageIcon(vImageURL);
            this.aOpponentPokemonImage.setIcon(new ImageIcon(vIcon.getImage().getScaledInstance(960, 904, java.awt.Image.SCALE_SMOOTH)));
        } else {
            ImageIcon vIcon = new ImageIcon(vImageURL);
            vIcon.getImage().flush();
            this.aOpponentPokemonImage.setIcon(vIcon);
        }
    }
}

} // showOpponentPokemonImage()

/**
 * Updates player HP bar.
 * @param pCurrent current HP
 * @param pMax max HP
 */
public void setPlayerHP(int pCurrent, int pMax) {
    this.aPlayerHPBar.setMaximum(pMax);
    this.aPlayerHPBar.setValue(pCurrent);
    this.aPlayerHPBar.setString("Your Pokemon: " + pCurrent + "/" + pMax);
} // setPlayerHP()

/**
 * Updates opponent HP bar.
 * @param pCurrent current HP
 * @param pMax max HP
 */
public void setOpponentHP(int pCurrent, int pMax) {
    this.aOpponentHPBar.setMaximum(pMax);
    this.aOpponentHPBar.setValue(pCurrent);
    this.aOpponentHPBar.setString("Opponent: " + pCurrent + "/" + pMax);
} // setOpponentHP()

/**
 * Gets attack button 1 for adding listeners.
 */
public JButton getAttack1Button() {
    return this.aAttack1Button;
}

```

```

private String aFilePath;
private Player aPlayer;
private Thread aThread;
private boolean aIsPlaying;

public MusicPlayer(String pFilePath) {
    this.aFilePath = pFilePath;
    this.aIsPlaying = false;
}

/**
 * Starts playing the MP3 in a loop on a background thread.
 */
public void playLoop() {
    this.aIsPlaying = true;
    this.aThread = new Thread() -> {
        while (this.aIsPlaying) {
            try {
                FileInputStream vFIS = new FileInputStream(this.aFilePath);
                this.aPlayer = new Player(vFIS);
                this.aPlayer.play();
                this.aPlayer.close();
            } catch (Exception e) {
                System.out.println("Error playing music: " + e.getMessage());
                this.aIsPlaying = false;
            }
        }
    });
    this.aThread.setDaemon(true);
    this.aThread.start();
}

/**
 * Stops the music.
 */
public void stop() {
    this.aIsPlaying = false;
    if (this.aPlayer != null) {
        this.aPlayer.close();
    }
}

```

III. Guide Utilisateur

Pour jouer au jeu :

Créer une instance de Game ou exécuter la commande java Game.

Utiliser les commandes: go [direction](north; east; south; west; up; down) , back, take [item], drop [item], name [new name], test [test filename]. look, cashin [item], help, quit, inventory, save [file name], load [filename], battle [trainer name].

IV. Déclaration Anti-Plagiat

Moi, Barnabé Jouanard, déclare n'avoir copié aucune ligne de code d'une source externe.